AWT Important qn All 5 Units [NMIT QB]

Module 1 – XHTML, CSS, HTML5, Bootstrap

1. Define XHTML and explain its basic rules. Write the structure of an XHTML document and create a simple webpage with heading, paragraph, list, and table.

XHTML

XHTML stands for Extensible HyperText Markup Language.
It is a stricter and cleaner version of HTML based on XML rules.

Basic Rules of XHTML

1. Every tag must be closed.
2. Tags must be in lowercase.
3. Elements must be properly nested.
4. Attribute values must be in quotes.
5. Empty tags must be self-closing.

Structure of XHTML Document

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Page Title</title>
  <meta charset="utf-8" />
</head>

<body>
  Content goes here
</body>
</html>
```

XHTML Webpage Example

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>XHTML Example</title>
  <meta charset="utf-8" />
</head>
```

```
<body>

  <h1>Welcome to XHTML</h1>

  <p>This is a simple XHTML webpage.</p>

  <ul>
    <li>HTML</li>
    <li>CSS</li>
    <li>JavaScript</li>
  </ul>

  <table border="1">
    <tr>
      <th>Name</th>
      <th>Course</th>
    </tr>

    <tr>
      <td>Ram</td>
      <td>MCA</td>
    </tr>
  </table>

</body>
</html>
```

2. Explain XHTML form elements and attributes. Create a feedback form with text box, radio button, checkbox, and submit button.

XHTML Form Elements and Attributes

Forms in XHTML are used to collect user input such as name, feedback, gender, and hobbies.

Common Form Elements

Element	Purpose
<form>	Creates form
<input>	Input field

<textarea>	Multi-line text
<radio>	Select one option
<checkbox>	Select multiple options
<submit>	Submit form

Common Form Attributes

Attribute	Purpose
type	Specifies input type
name	Name of field
value	Default value
action	Specifies form destination
method	Specifies GET or POST

XHTML Feedback Form

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Feedback Form</title>
  <meta charset="utf-8" />
</head>

<body>

<h2>Feedback Form</h2>

<form action="#" method="post">

  Name:
  <input type="text" name="uname" />
  <br /><br />

  Gender:
  <input type="radio" name="gender" value="male" /> Male
  <input type="radio" name="gender" value="female" /> Female
  <br /><br />
```

```
  Hobbies:
  <input type="checkbox" name="hobby" value="music" /> Music
  <input type="checkbox" name="hobby" value="sports" /> Sports
  <br /><br />

  <input type="submit" value="Submit" />

</form>

</body>
</html>
```

3. Explain ordered list, unordered list, and table tags in XHTML. Create a webpage to display event schedule.

Ordered List, Unordered List, and Table Tags in XHTML

Ordered List

Ordered list is used to display items in numbered format.

Tag used:

Example:

```
<ol>
  <li>HTML</li>
  <li>CSS</li>
</ol>
```

Unordered List

Unordered list is used to display items with bullets.

Tag used:

Example:

```
<ul>
  <li>Music</li>
  <li>Sports</li>
</ul>
```

Table Tags

Tables are used to display data in rows and columns.

Tag	Purpose
<table>	Creates table
<tr>	Table row
<th>	Table heading
<td>	Table data

Example:

```
<table border="1">
  <tr>
    <th>Name</th>
    <th>Course</th>
  </tr>

  <tr>
    <td>Ram</td>
    <td>MCA</td>
  </tr>
</table>
```

XHTML Webpage for Event Schedule

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
  <title>Event Schedule</title>
  <meta charset="utf-8" />
</head>

<body>
```

```
<h2>College Event Schedule</h2>

<h3>Events</h3>

<ol>
  <li>Technical Quiz</li>
  <li>Coding Contest</li>
  <li>Project Presentation</li>
</ol>

<h3>Facilities</h3>

<ul>
  <li>WiFi</li>
  <li>Food Court</li>
  <li>Parking</li>
</ul>

<h3>Schedule Table</h3>

<table border="1">

  <tr>
    <th>Event</th>
    <th>Time</th>
    <th>Venue</th>
  </tr>

  <tr>
    <td>Technical Quiz</td>
    <td>10:00 AM</td>
    <td>Hall A</td>
  </tr>

  <tr>
    <td>Coding Contest</td>
    <td>1:00 PM</td>
    <td>Lab 1</td>
  </tr>

</table>

</body>
</html>
```

4. What is CSS? Explain different types of CSS with examples.

CSS

CSS stands for Cascading Style Sheets.
It is used to style and design HTML/XHTML webpages.

CSS is used for:

- Colors
- Fonts
- Spacing
- Layout
- Alignment

Types of CSS

1. Inline CSS
2. Internal CSS
3. External CSS

1. Inline CSS

Inline CSS is written inside the HTML tag using style attribute.

Example:

```
<p style="color:red;">
  Welcome to CSS
</p>
```

2. Internal CSS

Internal CSS is written inside <style> tag in the <head> section.

Example:

```
<!DOCTYPE html>
<html>
<head>

<style>
```

```
p {
  color: blue;
}
</style>

</head>

<body>

<p>Internal CSS Example</p>

</body>
</html>
```

3. External CSS

External CSS is written in a separate .css file and linked using <link> tag.

CSS File (style.css)

```
p {
  color: green;
}
```

HTML File

```
<!DOCTYPE html>
<html>
<head>

<link rel="stylesheet" href="style.css" />

</head>

<body>

<p>External CSS Example</p>

</body>
</html>
```

5. Explain CSS Box Model with neat diagram.

CSS Box Model

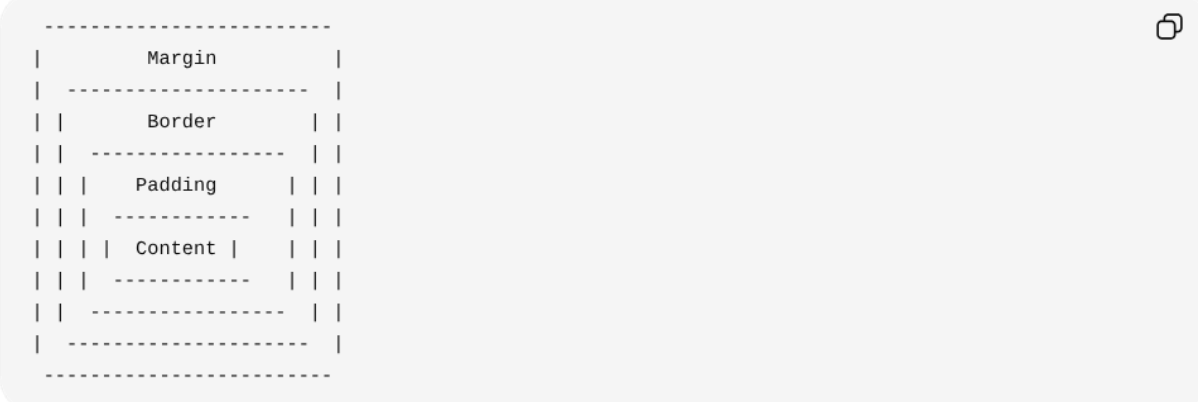
CSS Box Model describes how elements are displayed with content, padding, border, and margin around them.

Every HTML element is treated as a rectangular box.

Parts of Box Model

1. Content
2. Padding
3. Border
4. Margin

Diagram of CSS Box Model



Explanation

Part	Purpose
Content	Actual text or image
Padding	Space around content
Border	Border around padding
Margin	Space outside border

Example Program

<!DOCTYPE html>
<html>

```
<head>

<style>
div {
  width: 200px;
  padding: 20px;
  border: 5px solid black;
  margin: 30px;
}
</style>

</head>

<body>

<div>
  CSS Box Model Example
</div>

</body>
</html>
```

6. Explain CSS selectors and property–value pairs. Write CSS syntax to style heading, paragraph, and image.

CSS Selectors and Property–Value Pairs

CSS selectors are used to select HTML elements for styling.

A property–value pair defines the style to be applied.

Syntax:

```
selector {
  property: value;
}
```

Example:

```
p {
  color: red;
}
```

Here:

- p → selector
- color → property
- red → value

Types of CSS Selectors

Selector	Purpose
Element Selector	Selects HTML elements
ID Selector	Selects element using id
Class Selector	Selects elements using class

Element Selector

```
h1 {
  color: blue;
}
```

ID Selector

```
#title {
  color: green;
}
```

Class Selector

```
.note {
  color: red;
}
```

CSS Syntax to Style Heading, Paragraph, and Image

```
<!DOCTYPE html>
<html>
<head>

<style>

h1 {
```

```
  color: blue;
  text-align: center;
}

p {
  color: green;
  font-size: 20px;
}

img {
  width: 200px;
  border: 2px solid black;
}

</style>

</head>

<body>

<h1>Welcome to CSS</h1>

<p>This is paragraph styling.</p>

<img src="pic.jpg" alt="Image" />

</body>
</html>
```

7. Explain CSS properties used for spacing, alignment, color, and font. Design a product card layout.

CSS Properties for Spacing, Alignment, Color, and Font

CSS properties are used to style webpage elements.

Spacing Properties

Property	Purpose
margin	Space outside element
padding	Space inside element

Example:

margin: 20px;
padding: 10px;

Alignment Properties

Property	Purpose
text-align	Aligns text
vertical-align	Vertical alignment

Example:

text-align: center;

Color Properties

Property	Purpose
color	Text color
background-color	Background color

Example:

color: white;
background-color: blue;

Font Properties

Property	Purpose

font-size	Text size
font-family	Font style
font-weight	Bold text

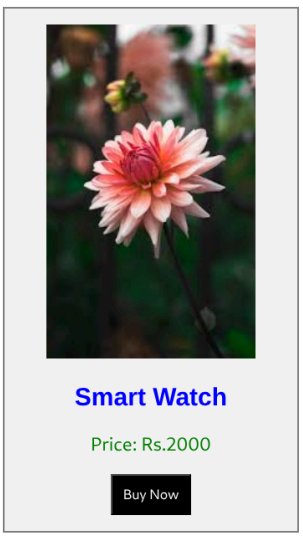
Example:

font-size: 20px;
font-family: Arial;
font-weight: bold;

Product Card Layout

```
<!DOCTYPE html>
<html>
<head>

<style>

.card {
  width: 250px;
  border: 2px solid gray;
  padding: 15px;
  margin: 20px;
  text-align: center;
  background-color: #f2f2f2;
}

.card img {
  width: 200px;
}

.card h2 {
  color: blue;
  font-family: Arial;
}

.card p {
  color: green;
  font-size: 18px;
}

button {
```

```
    padding: 10px;
    background-color: black;
    color: white;
}

</style>

</head>

<body>

<div class="card">

    <img src="product.jpg" alt="Product" />

    <h2>Smart Watch</h2>

    <p>Price: Rs.2000</p>

    <button>Buy Now</button>

</div>

</body>
</html>
```



8. Define HTML5 and explain its key features.

HTML5

HTML5 is the latest version of HTML used to create modern webpages and web applications.

It provides:

- Better multimedia support
- New semantic tags
- Improved forms
- Responsive web design support

Key Features of HTML5

1. New semantic elements
2. Audio and video support
3. Canvas and SVG graphics
4. Improved form controls
5. Responsive web design support
6. Local storage support
7. Cross-platform compatibility

New Semantic Elements

HTML5 introduced semantic tags such as:

Tag	Purpose
<header>	Page header
<nav>	Navigation links
<section>	Section content
<article>	Independent content
<footer>	Footer section

Example:

```
<header>
  Website Header
</header>
```

Audio and Video Support

HTML5 supports multimedia without external plugins.

Example:

```
<audio controls>
  <source src="song.mp3" type="audio/mp3" />
</audio>

<video width="300" controls>
  <source src="movie.mp4" type="video/mp4" />
</video>
```

Canvas and SVG

Canvas and SVG are used for graphics and animations.

Example:

```
<canvas width="200" height="100"></canvas>
```

Improved Forms

HTML5 introduced new input types.

Example:

```
<input type="email" />
<input type="date" />
```

Responsive Web Design

HTML5 supports responsive webpages using viewport.

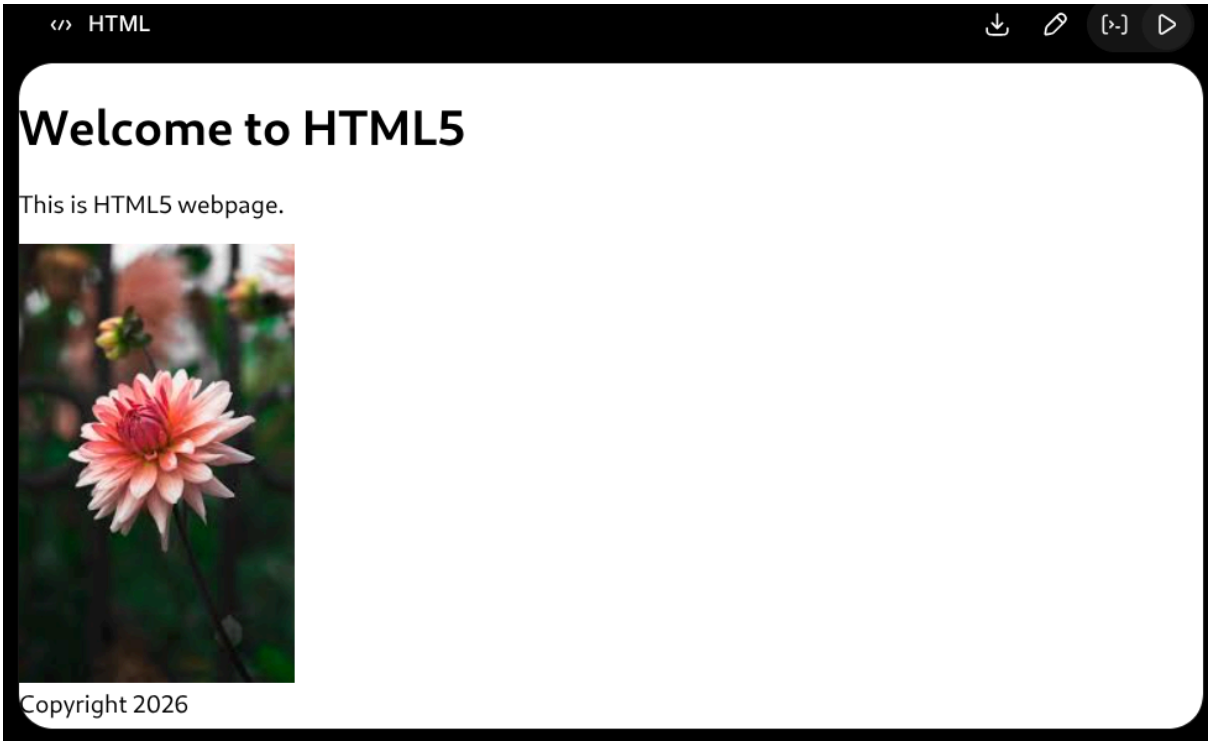
Example:

```
<meta name="viewport"
content="width=device-width, initial-scale=1.0" />
```

Simple HTML5 Webpage

```
<!DOCTYPE html>
```

```
<html>
<head>

<title>HTML5 Example</title>

<meta charset="utf-8" />

<meta name="viewport"
content="width=device-width, initial-scale=1.0" />

</head>

<body>

<header>
  <h1>Welcome to HTML5</h1>
</header>

<section>
  <p>This is HTML5 webpage.</p>

  <img
src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQQsXqMb8PFD4fJF6ZO4n1bnf3t0ds9VEnhow&s" alt="Image" />
</section>

<footer>
  Copyright 2026
</footer>

</body>
</html>
```



9. Explain HTML5 semantic tags with examples. Write HTML5 structure for a blog homepage.

HTML5 Semantic Tags

Semantic tags in HTML5 clearly describe the meaning of webpage content.

They improve:

- Readability
- SEO
- Webpage structure

Common Semantic Tags

Tag	Purpose
<header>	Header section
<nav>	Navigation links
<section>	Section content
<article>	Independent content
<aside>	Sidebar content

<footer>	Footer section

Examples

```
<header>
  Website Header
</header>

<nav>
  Home About Contact
</nav>

<section>
  Section Content
</section>

<article>
  Blog Post
</article>

<footer>
  Copyright 2026
</footer>
```

HTML5 Blog Homepage Structure

```
<!DOCTYPE html>
<html>
<head>

<title>Blog Homepage</title>

<meta charset="utf-8" />

<meta name="viewport"
content="width=device-width, initial-scale=1.0" />

</head>

<body>

<header>
  <h1>My Blog</h1>
```

```
</header>

<nav>
  <a href="#">Home</a> |
  <a href="#">Articles</a> |
  <a href="#">Contact</a>
</nav>

<section>

  <article>

    <h2>HTML5 Introduction</h2>

    <p>
      HTML5 is used for modern web development.
    </p>

    <img
src="https://encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcQQsXqMb8PFD4fJF6ZO4n1bnf3t0ds9VEnhow&s" alt="Blog Image" />

  </article>

</section>

<aside>
  Latest Posts
</aside>

<footer>
  Copyright 2026
</footer>

</body>
</html>
```

10. Explain multimedia support in HTML5. Write syntax to embed audio and video.

Multimedia Support in HTML5

HTML5 provides multimedia support using audio and video elements without external plugins.

HTML5 supports:

- Audio playback
- Video playback
- Multimedia controls

Audio Tag

The <audio> tag is used to embed audio files in webpage.

Syntax:

```
<audio controls>
  <source src="song.mp3" type="audio/mp3" />
</audio>
```

Attributes of Audio Tag

Attribute	Purpose
controls	Displays audio controls
autoplay	Plays audio automatically
loop	Repeats audio
muted	Mutes audio

Video Tag

The <video> tag is used to embed videos in webpage.

Syntax:

```
<video width="400" controls>
  <source src="movie.mp4" type="video/mp4" />
</video>
```

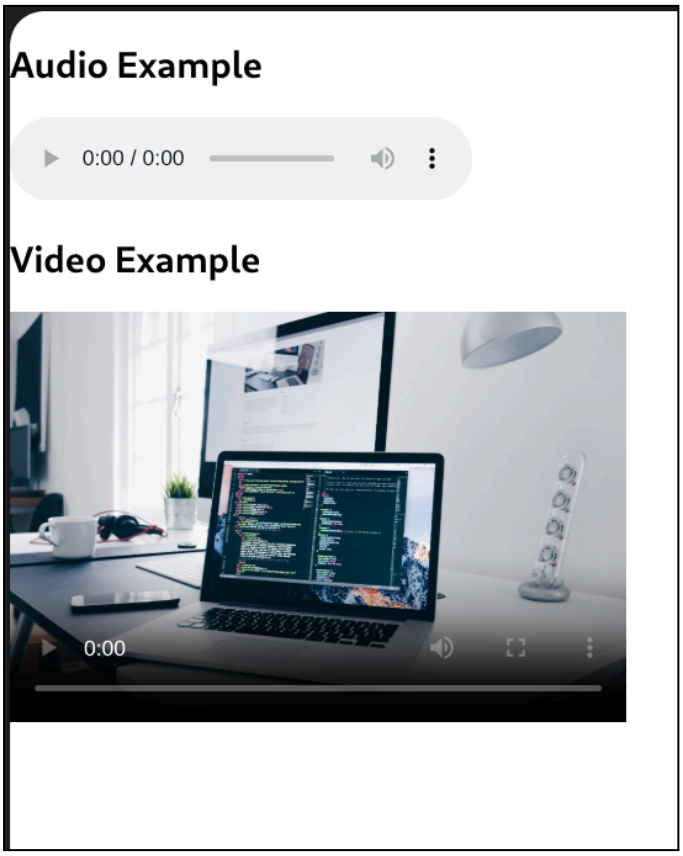
Attributes of Video Tag

Attribute	Purpose
controls	Displays video controls
width	Sets video width
height	Sets video height
autoplay	Plays video automatically
loop	Repeats video

HTML5 Multimedia Example

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML5 Multimedia</title>
</head>

<body>

<h2>Audio Example</h2>
```

```
<audio controls>
  <source src="song.mp3" type="audio/mp3" />
</audio>

<h2>Video Example</h2>

<video width="400" controls
poster="https://images.unsplash.com/photo-1498050108023-c5249f4df085">
  <source src="movie.mp4" type="video/mp4" />
</video>

</body>
</html>
```



11. What is responsive web design? Why is it important?

Responsive Web Design

Responsive Web Design (RWD) is a web design approach in which webpages automatically adjust according to different screen sizes and devices.

A responsive webpage works properly on:

- Mobile phones
- Tablets
- Laptops
- Desktop computers

Importance of Responsive Web Design

1. Improves user experience
2. Supports all screen sizes
3. Makes website mobile-friendly
4. Reduces scrolling and zooming
5. Improves website accessibility
6. Easier website maintenance

Features of Responsive Web Design

- Flexible layouts
- Flexible images
- CSS media queries
- Viewport support

Viewport Meta Tag

The viewport tag helps webpages adjust to device width.

Syntax:

```
<meta name="viewport"
content="width=device-width, initial-scale=1.0" />
```

Example of Responsive Webpage

```
<!DOCTYPE html>
<html>
<head>

<title>Responsive Web Design</title>

<meta name="viewport"
content="width=device-width, initial-scale=1.0" />

<style>

body {
```

```
    font-family: Arial;
}

img {
    width: 100%;
    height: auto;
}

</style>

</head>

<body>

<h1>Responsive Webpage</h1>

<img src="https://images.unsplash.com/photo-1498050108023-c5249f4df085"
alt="Responsive Image" />

<p>
This webpage adjusts according to screen size.
</p>

</body>
</html>
```

12. What is Bootstrap? Explain its features and advantages.

Bootstrap

Bootstrap is a free front-end framework used to create responsive and mobile-friendly webpages easily.

It contains:

- CSS classes
- JavaScript components
- Responsive grid system

Bootstrap helps in faster web development.

Features of Bootstrap

1. Responsive design
2. Mobile-first approach
3. Ready-made CSS classes
4. Grid system for layouts
5. Predefined components
6. Easy customization
7. Cross-browser compatibility

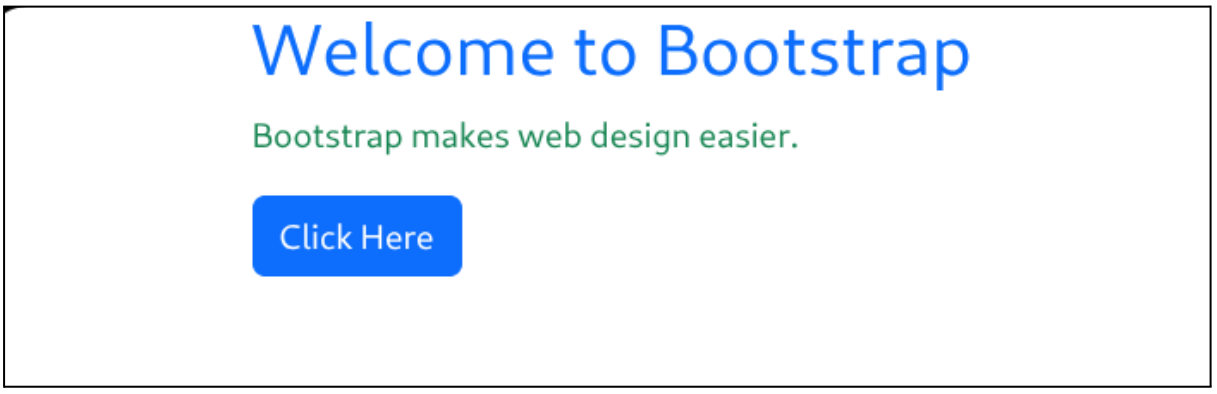
Advantages of Bootstrap

1. Faster webpage development
2. Easy to learn and use
3. Creates responsive webpages
4. Reduces CSS coding
5. Attractive user interface
6. Supports all modern browsers

Example of Bootstrap

```
<!DOCTYPE html>
<html>
<head>

  <title>Bootstrap Example</title>

  <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
```

```
</head>

<body>

  <div class="container">

    <h1 class="text-primary">
      Welcome to Bootstrap
    </h1>

    <p class="text-success">
      Bootstrap makes web design easier.
    </p>

    <button class="btn btn-primary">
      Click Here
    </button>

  </div>

</body>
</html>
```



13. Explain Bootstrap grid system and containers. Write syntax to display four product cards.

Bootstrap Grid System and Containers

Bootstrap Grid System is used to create responsive webpage layouts using rows and columns.

Bootstrap divides the webpage into 12 columns.

Bootstrap Containers

Containers are used to wrap webpage content.

Class	Purpose
container	Fixed width container
container-fluid	Full width container

Example:

```
<div class="container">
  Content
</div>
```

Bootstrap Grid System

Grid system uses:

- row
- col

Example:

```
<div class="row">

  <div class="col-md-6">
    Column 1
  </div>

  <div class="col-md-6">
    Column 2
  </div>

</div>
```

Here:

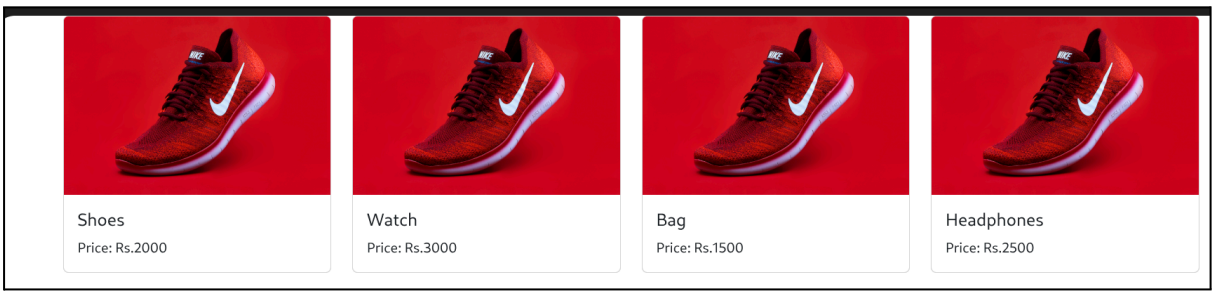
- row creates horizontal row
- col-md-6 occupies 6 columns out of 12

Bootstrap Product Cards Example

```
<!DOCTYPE html>
<html>
<head>

  <title>Bootstrap Product Cards</title>

  <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">

</head>

<body>

<div class="container">

  <div class="row">

    <div class="col-md-3">

      <div class="card">

        <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
        class="card-img-top"
        alt="Product">

        <div class="card-body">

          <h5 class="card-title">
            Shoes
          </h5>

          <p class="card-text">
            Price: Rs.2000
          </p>

        </div>

      </div>

    </div>

    <div class="col-md-3">

      <div class="card">
```

```
      <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
      class="card-img-top"
      alt="Product">

      <div class="card-body">

        <h5 class="card-title">
          Watch
        </h5>

        <p class="card-text">
          Price: Rs.3000
        </p>

      </div>

    </div>

  </div>

  <div class="col-md-3">

    <div class="card">

      <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
      class="card-img-top"
      alt="Product">

      <div class="card-body">

        <h5 class="card-title">
          Bag
        </h5>

        <p class="card-text">
          Price: Rs.1500
        </p>

      </div>

    </div>

  </div>
```

```
      <div class="col-md-3">

        <div class="card">

          <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
          class="card-img-top"
          alt="Product">

          <div class="card-body">

            <h5 class="card-title">
              Headphones
            </h5>

            <p class="card-text">
              Price: Rs.2500
            </p>

          </div>

        </div>

      </div>

    </div>

</div>

</body>
</html>
```



14. Explain Bootstrap components such as navbar, cards, and buttons.

Bootstrap Components: Navbar, Cards, and Buttons

Bootstrap provides ready-made components to design responsive webpages easily.

Navbar

Navbar is used to create navigation menus for websites.

Features:

- Responsive menu
- Navigation links
- Website title or logo

Example:

```
<nav class="navbar navbar-dark bg-dark">

  <div class="container-fluid">

    <a class="navbar-brand" href="#">
      My Website
    </a>

  </div>

</nav>
```

Cards

Cards are flexible containers used to display content such as:

- Products
- Profiles
- Blog posts

Example:

```
<div class="card" style="width:250px;">

  <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
  class="card-img-top"
  alt="Product">

  <div class="card-body">

    <h5 class="card-title">
```

```
      Shoes
    </h5>

    <p class="card-text">
      Price: Rs.2000
    </p>

  </div>

</div>
```

Buttons

Buttons are used for user actions such as submit, login, and navigation.

Example:

```
<button class="btn btn-primary">
  Click Here
</button>
```

Types of Bootstrap Buttons

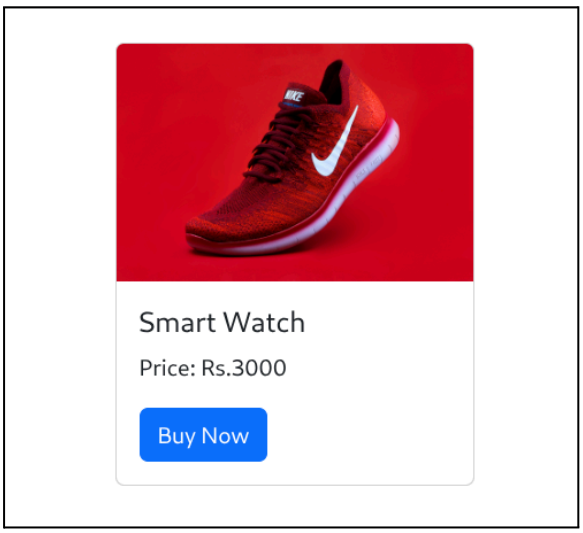
Class	Color
btn-primary	Blue
btn-success	Green
btn-danger	Red
btn-warning	Yellow

Combined Example

```
<!DOCTYPE html>
<html>
<head>

  <title>Bootstrap Components</title>

  <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">
```

```
</head>

<body>

<nav class="navbar navbar-dark bg-dark">

  <div class="container-fluid">

    <a class="navbar-brand" href="#">
      My Store
    </a>

  </div>

</nav>

<div class="container mt-4">

  <div class="card" style="width:250px;">

    <img src="https://images.unsplash.com/photo-1542291026-7eec264c27ff"
    class="card-img-top"
    alt="Product">

    <div class="card-body">

      <h5 class="card-title">
        Smart Watch
      </h5>

      <p class="card-text">
        Price: Rs.3000
      </p>

      <button class="btn btn-primary">
        Buy Now
      </button>

    </div>

  </div>

</div>
```

</body>
</html>



15. Explain how XHTML, CSS, HTML5 semantic tags, and Bootstrap can be used to build a startup website.

Using XHTML, CSS, HTML5 Semantic Tags, and Bootstrap to Build a Startup Website

XHTML

XHTML is used to create the structure of the startup website.

It provides:

- Proper webpage structure
- Forms
- Tables
- Lists

Example:

<h1>Startup Company</h1>

Welcome to our website.

CSS

CSS is used to style the startup website.

It helps in:

- Colors
- Fonts
- Spacing
- Layout design

Example:

```
h1 {
  color: blue;
  text-align: center;
}
```

HTML5 Semantic Tags

HTML5 semantic tags organize webpage content clearly.

Common semantic tags:

Tag	Purpose
<header>	Website header
<nav>	Navigation menu
<section>	Main section
<article>	Content area
<footer>	Footer

Example:

```
<header>
  Startup Website
</header>

<nav>
  Home About Contact
</nav>
```

Bootstrap

Bootstrap is used to make the startup website responsive and attractive.

Bootstrap provides:

- Grid system
- Navbar
- Buttons
- Cards
- Forms

Example:

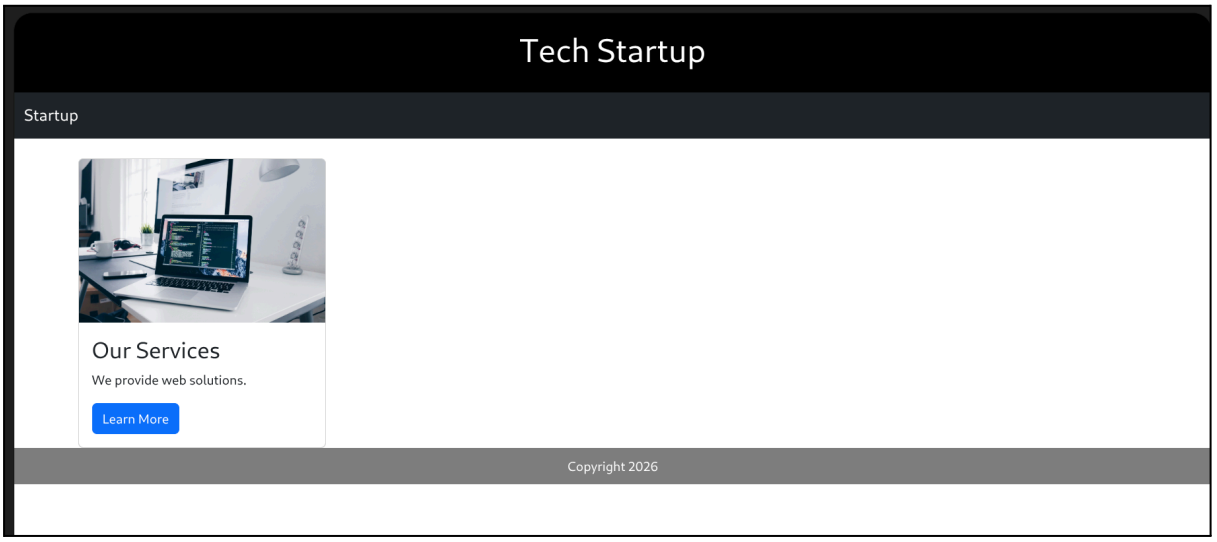
```
<button class="btn btn-primary">
  Get Started
</button>
```

Startup Website Example

```
<!DOCTYPE html>
<html>
<head>

  <title>Startup Website</title>

  <meta charset="utf-8" />

  <meta name="viewport"
  content="width=device-width, initial-scale=1.0" />

  <link rel="stylesheet"
  href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css">

  <style>

    body {
      font-family: Arial;
    }

    header {
      background-color: black;
      color: white;
      text-align: center;
      padding: 20px;
    }
```

```
    footer {
      background-color: gray;
      color: white;
      text-align: center;
      padding: 10px;
    }

  </style>

</head>

<body>

<header>
  <h1>Tech Startup</h1>
</header>

<nav class="navbar navbar-expand-lg navbar-dark bg-dark">

  <div class="container-fluid">

    <a class="navbar-brand" href="#">
      Startup
    </a>

  </div>

</nav>

<section class="container mt-4">

  <div class="card" style="width:300px;">

    <img src="https://images.unsplash.com/photo-1498050108023-c5249f4df085"
    class="card-img-top"
    alt="Startup">

    <div class="card-body">

      <h3 class="card-title">
        Our Services
      </h3>
```

```
        <p class="card-text">
          We provide web solutions.
        </p>

        <button class="btn btn-primary">
          Learn More
        </button>

      </div>

    </div>

</section>

<footer>
  Copyright 2026
</footer>

</body>
</html>
```



Module 2 – PHP

1. Define PHP and explain its features. Write the basic syntax of a PHP program.

PHP

PHP stands for Hypertext Preprocessor.

PHP is a server-side scripting language used to create dynamic web applications.

PHP is mainly used for:

- Form handling
- Database connectivity
- Dynamic webpages

Features of PHP

1. Open source
2. Server-side scripting language
3. Supports database connectivity
4. Easy to embed in HTML
5. Platform independent
6. Supports dynamic webpages
7. Simple and easy to learn

Basic Syntax of PHP Program

PHP code is written inside:

```
<?php
  // PHP code
?>
```

Basic PHP Program

```
<!DOCTYPE html>
<html>
<head>

  <title>PHP Example</title>

</head>

<body>

<?php

  echo "Welcome to PHP";

?>
```

</body>
</html>

Explanation

Statement	Purpose
<?php ?>	PHP opening and closing tags
echo	Displays output
;	Ends statement

2. Explain PHP variables and data types with examples. Store product details and display them.

PHP Variables and Data Types

In PHP, variables are used to store data.

Every variable in PHP starts with $ symbol.

Example:

$name = "Ram";

Rules for Variables

1. Variable name starts with $
2. Variable names are case sensitive
3. Variables need not be declared before use

PHP Data Types

Data Type	Example
Integer	10
Double	10.5
String	"Hello"
Boolean	true or false

Array	array()

Example of Variables and Data Types

```
<?php

   $name = "Laptop";
   $price = 45000;
   $rating = 4.5;
   $available = true;

?>
```

PHP Program to Store and Display Product Details

```
<!DOCTYPE html>
<html>
<head>

   <title>Product Details</title>

</head>

<body>

<?php

   $product = "Smart Phone";
   $price = 25000;
   $brand = "Samsung";
   $stock = true;

   echo "<h2>Product Details</h2>";

   echo "Product Name: " . $product . "<br>";

   echo "Price: Rs." . $price . "<br>";

   echo "Brand: " . $brand . "<br>";

   echo "Available: " . $stock;

?>
```

</body>
</html>

3. Explain different types of operators in PHP. Write a program to calculate total bill amount.

PHP Operators

Operators in PHP are used to perform operations on variables and values.

Types of Operators in PHP

Operator Type	Purpose
Arithmetic Operators	Mathematical calculations
Assignment Operators	Assign values
Comparison Operators	Compare values
Logical Operators	Combine conditions
Increment/Decrement Operators	Increase or decrease value

Arithmetic Operators

Operator	Purpose
+	Addition
-	Subtraction
*	Multiplication
/	Division
%	Modulus

Example:

```
$a = 10;
$b = 5;

echo $a + $b;
```

Assignment Operator

$x = 100;

Comparison Operators

Operator	Purpose
==	Equal to
!=	Not equal to
>	Greater than
<	Less than

Logical Operators

Operator	Purpose
&&	AND
`	
!	NOT

PHP Program to Calculate Total Bill Amount

```
<!DOCTYPE html>
<html>
<head>

  <title>Total Bill</title>

</head>

<body>

<?php

  $product1 = 500;
  $product2 = 700;
  $product3 = 300;
```

```
    $total = $product1 + $product2 + $product3;

    echo "<h2>Bill Details</h2>";

    echo "Product 1 Price: Rs." . $product1 . "<br>";

    echo "Product 2 Price: Rs." . $product2 . "<br>";

    echo "Product 3 Price: Rs." . $product3 . "<br><br>";

    echo "Total Bill Amount: Rs." . $total;

?>

</body>
</html>
```

4. Explain control statements in PHP. Write a program to check discount eligibility.

Control Statements in PHP

Control statements in PHP are used to control the flow of program execution.

Types of Control Statements

Statement	Purpose
if	Executes block if condition is true
if-else	Executes one block among two
elseif	Checks multiple conditions
switch	Selects one block from many
for	Repeats block fixed number of times
while	Repeats while condition is true

if Statement

Syntax:

```
if(condition) {
   statements;
}
```

if-else Statement

Syntax:

```
if(condition) {
   statements;
}
else {
   statements;
}
```

PHP Program to Check Discount Eligibility

```
<!DOCTYPE html>
<html>
<head>

   <title>Discount Eligibility</title>

</head>

<body>

<?php

   $amount = 6000;

   echo "<h2>Shopping Details</h2>";

   echo "Purchase Amount: Rs." . $amount . "<br><br>";

   if($amount >= 5000) {

      echo "Customer is Eligible for Discount";

   }
   else {

      echo "Customer is Not Eligible for Discount";
```

```
    }

?>

</body>
</html>
```

5. Explain indexed, associative, and multidimensional arrays in PHP.

Arrays in PHP

Arrays in PHP are used to store multiple values in a single variable.

Types of Arrays in PHP

1. Indexed Array
2. Associative Array
3. Multidimensional Array

Indexed Array

Indexed array stores values using numeric indexes starting from 0.

Example:

```
<?php

    $colors = array("Red", "Blue", "Green");

    echo $colors[0];

?>
```

Associative Array

Associative array stores values using named keys.

Example:

```
<?php

    $student = array(
```

```
    "name" => "Ram",
    "course" => "MCA",
    "age" => 22
  );

  echo $student["name"];

?>
```

Multidimensional Array

Multidimensional array contains one or more arrays inside another array.

Example:

```
<?php

  $students = array(

    array("Ram", "MCA", 22),
    array("Sam", "BCA", 21)

  );

  echo $students[0][0];

?>
```

PHP Program Using All Arrays

```
<!DOCTYPE html>
<html>
<head>

  <title>PHP Arrays</title>

</head>

<body>

<?php

  $products = array("Laptop", "Mobile", "Watch");
```

```
  echo "<h3>Indexed Array</h3>";
  echo $products[1];

  $student = array(
    "name" => "Anu",
    "course" => "MCA"
  );

  echo "<h3>Associative Array</h3>";
  echo $student["course"];

  $marks = array(

    array("Ram", 85),
    array("Sam", 90)

  );

  echo "<h3>Multidimensional Array</h3>";
  echo $marks[1][0];

?>

</body>
</html>
```

6. Explain user-defined functions in PHP. Write a function to calculate simple interest or total marks.

User-Defined Functions in PHP

User-defined functions in PHP are functions created by the programmer to perform specific tasks.

Functions help to:

- Reuse code
- Reduce program size
- Improve readability

Syntax of Function

```
function functionName() {

   statements;

}
```

Function with Parameters

```
function add($a, $b) {

   return $a + $b;

}
```

PHP Program to Calculate Simple Interest

```
<!DOCTYPE html>
<html>
<head>

   <title>Simple Interest</title>

</head>

<body>

<?php

   function simpleInterest($p, $r, $t) {

      $si = ($p * $r * $t) / 100;

      return $si;

   }

   $result = simpleInterest(10000, 5, 2);

   echo "<h2>Simple Interest</h2>";

   echo "Simple Interest = Rs." . $result;

?>
```

```
</body>
</html>
```

PHP Program to Calculate Total Marks

```
<!DOCTYPE html>
<html>
<head>

  <title>Total Marks</title>

</head>

<body>

<?php

  function totalMarks($m1, $m2, $m3) {

    $total = $m1 + $m2 + $m3;

    return $total;

  }

  $result = totalMarks(85, 90, 80);

  echo "<h2>Total Marks</h2>";

  echo "Total Marks = " . $result;

?>

</body>
</html>
```

7. Explain form handling in PHP using GET and POST methods.

Form Handling in PHP

Form handling in PHP is used to collect and process user input from HTML forms.

PHP mainly uses two methods:

1. GET
2. POST

GET Method

GET method sends data through URL.

Features:

- Data visible in URL
- Limited data size
- Less secure

Example URL:

page.php?name=Ram

Syntax of GET Method

```
<form method="get">
```

PHP GET Example

```
<!DOCTYPE html>
<html>
<body>

<form method="get">

  Name:
  <input type="text" name="uname">

  <input type="submit" value="Submit">

</form>

<?php

  if(isset($_GET['uname'])) {

    echo "Welcome " . $_GET['uname'];
```

```
  }

?>

</body>
</html>
```

POST Method

POST method sends data securely.

Features:

- Data not visible in URL
- More secure
- Supports large data

Syntax of POST Method

```
<form method="post">
```

PHP POST Example

```
<!DOCTYPE html>
<html>
<body>

<form method="post">

  Name:
  <input type="text" name="uname">

  <input type="submit" value="Submit">

</form>

<?php

  if(isset($_POST['uname'])) {

    echo "Welcome " . $_POST['uname'];

  }
```

```
?>

</body>
</html>
```

Difference Between GET and POST

GET	POST
Data visible in URL	Data hidden
Less secure	More secure
Limited data	Large data supported
Faster	Slightly slower

8. Explain file handling in PHP. Create/open a file, write feedback, and display content.

File Handling in PHP

File handling in PHP is used to create, open, read, write, and close files.

PHP provides built-in functions for file operations.

Common File Functions

Function	Purpose
fopen()	Opens or creates file
fwrite()	Writes data to file
fread()	Reads file content
fclose()	Closes file

File Modes

Mode	Purpose
r	Read file
w	Write file

a	Append file

PHP Program to Create/Open File, Write Feedback, and Display Content

```
<!DOCTYPE html>
<html>
<head>

   <title>File Handling</title>

</head>

<body>

<?php

   $file = fopen("feedback.txt", "w");

   fwrite($file, "Excellent Service");

   fclose($file);

   $file = fopen("feedback.txt", "r");

   $content = fread($file, filesize("feedback.txt"));

   echo "<h2>Feedback Content</h2>";

   echo $content;

   fclose($file);

?>

</body>
</html>
```

9. Explain PHP–MySQL connectivity steps. Write structure to connect and insert records.

PHP–MySQL Connectivity

PHP can connect with MySQL to store and retrieve data from database.

PHP–MySQL connectivity is used for:

- Registration forms
- Login systems
- Data storage

Steps for PHP–MySQL Connectivity

1. Create MySQL database
2. Create table
3. Connect PHP with MySQL
4. Write SQL query
5. Execute query
6. Close connection

Syntax to Connect PHP with MySQL

```
$conn = mysqli_connect(
   "localhost",
   "root",
   "",
   "testdb"
);
```

Explanation

Parameter	Purpose
localhost	Server name
root	Username
""	Password
testdb	Database name

PHP Program to Connect and Insert Records

```
<!DOCTYPE html>
<html>
<head>

   <title>PHP MySQL Connection</title>
```

```
</head>

<body>

<?php

   $conn = mysqli_connect(
      "localhost",
      "root",
      "",
      "testdb"
   );

   if(!$conn) {

      die("Connection Failed");

   }

   $sql = "INSERT INTO students(name, course)
         VALUES('Ram', 'MCA')";

   if(mysqli_query($conn, $sql)) {

      echo "Record Inserted Successfully";

   }
   else {

      echo "Error";

   }

   mysqli_close($conn);

?>

</body>
</html>
```

10. Explain how PHP retrieves records from MySQL database.

Retrieving Records from MySQL using PHP

PHP retrieves records from MySQL using SQL SELECT query.

The retrieved data can be displayed on webpage.

Steps to Retrieve Records

1. Connect PHP with MySQL database
2. Write SELECT query
3. Execute query using mysqli_query()
4. Fetch records using mysqli_fetch_assoc()
5. Display records
6. Close connection

SELECT Query Syntax

$sql = "SELECT * FROM students";

Functions Used

Function	Purpose
mysqli_connect()	Connects database
mysqli_query()	Executes query
mysqli_fetch_assoc()	Fetches records
mysqli_close()	Closes connection

PHP Program to Retrieve Records

```
<!DOCTYPE html>
<html>
<head>

  <title>Retrieve Records</title>

</head>

<body>
```

```
<?php

  $conn = mysqli_connect(
    "localhost",
    "root",
    "",
    "testdb"
  );

  if(!$conn) {

    die("Connection Failed");

  }

  $sql = "SELECT * FROM students";

  $result = mysqli_query($conn, $sql);

  echo "<h2>Student Records</h2>";

  while($row = mysqli_fetch_assoc($result)) {

    echo "Name: " . $row["name"] . "<br>";

    echo "Course: " . $row["course"] . "<br><br>";

  }

  mysqli_close($conn);

?>

</body>
</html>
```

11. Define cookies and explain their purpose. Write syntax to create and retrieve cookie.

Cookies in PHP

Cookies are small text files stored on the user's computer by the browser.

PHP uses cookies to store user information temporarily.

Purpose of Cookies

1. Store user information
2. Remember login details
3. Track user activity
4. Maintain user preferences

Creating a Cookie

PHP uses setcookie() function to create cookies.

Syntax:

```
setcookie(
  "cookie_name",
  "value",
  time()+3600
);
```

Explanation

Parameter	Purpose
Cookie name	Name of cookie
Value	Data stored
time()+3600	Expiry time

Retrieving a Cookie

Cookies are retrieved using $_COOKIE.

Syntax:

```
$_COOKIE["cookie_name"]
```

PHP Program to Create and Retrieve Cookie

```
<!DOCTYPE html>
<html>
<head>
```

```
    <title>PHP Cookies</title>

</head>

<body>

<?php

    setcookie(
        "username",
        "Ram",
        time()+3600
    );

    if(isset($_COOKIE["username"])) {

        echo "Welcome "
        . $_COOKIE["username"];

    }
    else {

        echo "Cookie Not Found";

    }

?>

</body>
</html>
```

12. What is session management in PHP? Explain session creation and destruction.

Session Management in PHP

Session management in PHP is used to store user data on the server across multiple webpages.

Sessions are mainly used for:

- Login systems
- Shopping carts

- User authentication

Features of Sessions

1. Stores user data on server
2. More secure than cookies
3. Maintains user information between pages

Session Creation

PHP uses session_start() function to create a session.

Syntax:

session_start();

Creating Session Variable

$_SESSION["username"] = "Ram";

Retrieving Session Variable

echo $_SESSION["username"];

Session Destruction

PHP uses session_destroy() function to destroy session.

Syntax:

session_destroy();

PHP Program for Session Creation and Destruction

```
<!DOCTYPE html>
<html>
<head>

  <title>PHP Sessions</title>

</head>

<body>
```

```
<?php

  session_start();

  $_SESSION["username"] = "Ram";

  echo "<h2>Session Created</h2>";

  echo "Username: "
  . $_SESSION["username"];

  session_destroy();

?>

</body>
</html>
```

13. Explain pattern matching in PHP using regular expressions. Validate email / phone number.

Pattern Matching in PHP using Regular Expressions

Pattern matching in PHP is used to check whether a string matches a specified pattern.

PHP uses regular expressions for:

- Email validation
- Phone number validation
- Password checking

Regular Expression Function

PHP uses preg_match() function.

Syntax:

preg_match(pattern, string)

Email Validation

Example pattern for email:

/^[a-zA-Z0-9._%+-]+@[a-zA-Z.-]+\.[a-zA-Z]{2,}$/

Phone Number Validation

Example pattern for phone number:

/^[0-9]{10}$/

PHP Program for Email Validation

```
<!DOCTYPE html>
<html>
<head>

   <title>Email Validation</title>

</head>

<body>

<?php

   $email = "user@gmail.com";

   $pattern = "/^[a-zA-Z0-9._%+-]+@[a-zA-Z.-]+\.[a-zA-Z]{2,}$/";

   if(preg_match($pattern, $email)) {

      echo "Valid Email Address";

   }
   else {

      echo "Invalid Email Address";

   }

?>

</body>
</html>
```

PHP Program for Phone Number Validation

```
<!DOCTYPE html>
<html>
<head>

  <title>Phone Validation</title>

</head>

<body>

<?php

  $phone = "9876543210";

  $pattern = "/^[0-9]{10}$/";

  if(preg_match($pattern, $phone)) {

    echo "Valid Phone Number";

  }
  else {

    echo "Invalid Phone Number";

  }

?>

</body>
</html>
```

14. Explain file upload process in PHP. Write basic structure to upload file.

File Upload Process in PHP

File upload in PHP allows users to upload files from webpage to server.

Examples:

- Image upload
- PDF upload
- Document upload

Steps in File Upload Process

1. Create HTML form
2. Use enctype="multipart/form-data"
3. Select file using file input
4. Access file using $_FILES
5. Move uploaded file using move_uploaded_file()

Important Attributes

Attribute	Purpose
method="post"	Sends form data
enctype="multipart/form-data"	Supports file upload
type="file"	Selects file

Basic Structure for File Upload

```
<!DOCTYPE html>
<html>
<head>

  <title>File Upload</title>

</head>

<body>

<form method="post"
enctype="multipart/form-data">

  Select File:

  <input type="file" name="myfile">

  <br><br>
```

```
  <input type="submit"
  value="Upload">

</form>

<?php

  if(isset($_FILES['myfile'])) {

    $filename = $_FILES['myfile']['name'];

    $tempname = $_FILES['myfile']['tmp_name'];

    move_uploaded_file(
      $tempname,
      "uploads/" . $filename
    );

    echo "File Uploaded Successfully";

  }

?>

</body>
</html>
```

15. Explain how PHP can be used for form handling, validation, database storage, sessions, and file upload in an online shopping website.

PHP in Online Shopping Website

PHP is widely used in online shopping websites for dynamic operations such as form handling, validation, database storage, sessions, and file uploads.

Form Handling

PHP collects user input from forms.

Uses:

- Registration forms

- Login forms
- Order forms

Example:

$name = $_POST['username'];

Validation

PHP validates user input before processing data.

Uses:

- Email validation
- Password validation
- Phone number checking

Example:

```
if(empty($name)) {

  echo "Name Required";

}
```

Database Storage

PHP connects with MySQL to store customer and product details.

Uses:

- Store user accounts
- Store orders
- Store product details

Example:

```
$sql = "INSERT INTO users(name)
VALUES('Ram')";
```

Sessions

Sessions store user information across webpages.

Uses:

- Login management
- Shopping cart
- User authentication

Example:

```
session_start();

$_SESSION['user'] = "Ram";
```

File Upload

PHP uploads files from user system to server.

Uses:

- Product image upload
- Profile photo upload

Example:

```
move_uploaded_file(
   $tempname,
   "uploads/" . $filename
);
```

Example of Online Shopping Workflow

1. User fills registration form
2. PHP validates user input
3. User details stored in MySQL database
4. Session created after login
5. Product images uploaded using file upload
6. Shopping cart maintained using sessions

Sample PHP Structure

```
<?php

   session_start();

   $conn = mysqli_connect(
```

```
    "localhost",
    "root",
    "",
    "shopdb"
);

$name = $_POST['username'];

if(!empty($name)) {

    $_SESSION['user'] = $name;

    $sql = "INSERT INTO users(name)
        VALUES('$name')";

    mysqli_query($conn, $sql);

    echo "Registration Successful";

}
else {

    echo "Validation Failed";

}

?>
```

Module 3 – JavaScript & DOM

1. Define JavaScript and explain its features.

JavaScript

JavaScript is a client-side scripting language used to make webpages interactive and dynamic.

JavaScript is used for:

- Form validation
- Calculations
- Interactive webpages

- Dynamic content updates

Features of JavaScript

1. Lightweight scripting language
2. Object-based language
3. Interpreted language
4. Platform independent
5. Supports event handling
6. Used for dynamic webpages
7. Runs in web browsers

Advantages of JavaScript

1. Faster execution
2. Improves webpage interactivity
3. Reduces server load
4. Easy to learn and use

Basic JavaScript Syntax

JavaScript code is written inside <script> tag.

Syntax:

```
<script>

  JavaScript code

</script>
```

JavaScript Example

```
<!DOCTYPE html>
<html>
<head>

  <title>JavaScript Example</title>

</head>

<body>

<h2>JavaScript Demo</h2>
```

```
<script>

  document.write("Welcome to JavaScript");

</script>

</body>
</html>
```

2. Explain JavaScript variables and data types with examples.

JavaScript Variables and Data Types

Variables in JavaScript are used to store data values.

JavaScript variables can be declared using:

- var
- let
- const

Syntax of Variable

var name = "Ram";

Rules for Variables

1. Variable names are case sensitive
2. Variable name can contain letters, digits, _ and $
3. Variable should not start with number

JavaScript Data Types

Data Type	Example

Number	10
String	"Hello"
Boolean	true
Array	[1,2,3]
Object	{name:"Ram"}

Number Data Type

var age = 20;

String Data Type

var name = "Ram";

Boolean Data Type

var status = true;

Array Data Type

var colors = ["Red", "Blue", "Green"];

JavaScript Program Using Variables and Data Types

```
<!DOCTYPE html>
<html>
<head>

  <title>Variables and Data Types</title>

</head>

<body>

<script>

  var product = "Laptop";

  var price = 45000;
```

```
    var available = true;

    document.write("<h2>Product Details</h2>");

    document.write("Product: " + product + "<br>");

    document.write("Price: Rs." + price + "<br>");

    document.write("Available: " + available);

</script>

</body>
</html>
```

3. Explain functions in JavaScript. Write a function to calculate total marks.

Functions in JavaScript

Functions in JavaScript are reusable blocks of code used to perform specific tasks.

Functions help to:

- Reduce code repetition
- Improve readability
- Reuse code easily

Syntax of Function

```
function functionName() {

    statements;
```

```
}
```

Function with Parameters

```
function add(a, b) {

  return a + b;

}
```

JavaScript Function to Calculate Total Marks

```
<!DOCTYPE html>
<html>
<head>

  <title>Total Marks</title>

</head>

<body>

<script>

  function totalMarks(m1, m2, m3) {

    var total = m1 + m2 + m3;

    return total;

  }

  var result = totalMarks(85, 90, 80);

  document.write("<h2>Total Marks</h2>");

  document.write("Total Marks = " + result);

</script>

</body>
</html>
```

4. Explain arrays in JavaScript. Store marks and calculate total.

Arrays in JavaScript

Arrays in JavaScript are used to store multiple values in a single variable.

Array values are stored using indexes starting from 0.

Syntax of Array

var arrayName = [value1, value2, value3];

Example of Array

var colors = ["Red", "Blue", "Green"];

Accessing Array Elements

document.write(colors[0]);

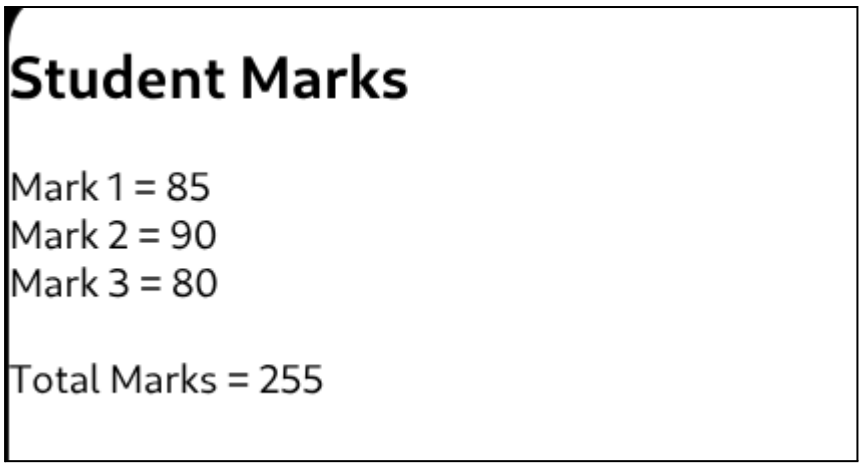
Features of Arrays

1. Stores multiple values
2. Uses index numbers
3. Easy data management

JavaScript Program to Store Marks and Calculate Total

```
<!DOCTYPE html>
<html>
<head>

  <title>JavaScript Arrays</title>

</head>
```

```
<body>

<script>

  var marks = [85, 90, 80];

  var total = marks[0] + marks[1] + marks[2];

  document.write("<h2>Student Marks</h2>");

  document.write("Mark 1 = " + marks[0] + "<br>");

  document.write("Mark 2 = " + marks[1] + "<br>");

  document.write("Mark 3 = " + marks[2] + "<br><br>");

  document.write("Total Marks = " + total);

</script>

</body>
</html>
```



5. Explain form validation in JavaScript.

Form Validation in JavaScript

Form validation in JavaScript is used to check whether user input is correct before submitting the form.

Validation helps to:

- Prevent empty fields
- Check valid data
- Improve form security

Types of Validation

1. Empty field validation
2. Email validation
3. Password validation
4. Number validation

Advantages of Form Validation

1. Reduces invalid data
2. Improves user experience
3. Saves server processing time

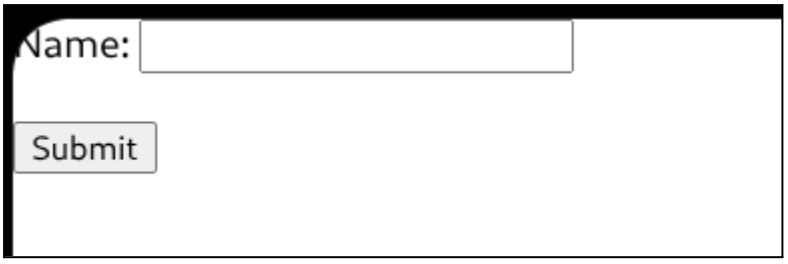
Syntax of Validation

```
if(condition) {

  statements;

}
```

JavaScript Form Validation Example

```
<!DOCTYPE html>
<html>
<head>

  <title>Form Validation</title>

<script>

function validateForm() {

  var name =
  document.forms["myform"]["uname"].value;

  if(name == "") {

    alert("Name must be filled");
```

```
    return false;
  }
}
</script>
</head>
<body>
<form name="myform"
onsubmit="return validateForm()">
  Name:
  <input type="text" name="uname">
  <br><br>
  <input type="submit" value="Submit">
</form>
</body>
</html>
```



6. Define event handling in JavaScript and explain different types of events.

Event Handling in JavaScript

Event handling in JavaScript is the process of responding to user actions on webpage.

Events occur when user interacts with webpage elements.

Examples:

- Clicking button
- Typing text
- Moving mouse

Types of Events in JavaScript

Event	Purpose
onclick	Triggered when element is clicked
onmouseover	Triggered when mouse moves over element
onmouseout	Triggered when mouse leaves element
onkeyup	Triggered when key is released
onload	Triggered when webpage loads
onsubmit	Triggered when form is submitted

onclick Event

```
<button onclick="show()">
  Click Here
</button>
```

onmouseover Event

```
<p onmouseover="alert('Mouse Over')">
  Move Mouse Here
</p>
```

onload Event

```
<body onload="welcome()">
```

JavaScript Event Handling Example

```
<!DOCTYPE html>
<html>
<head>
```

```
  <title>Event Handling</title>

<script>

function display() {

  alert("Button Clicked");

}

</script>

</head>

<body>

<h2>JavaScript Events</h2>

<button onclick="display()">
  Click Here
</button>

</body>
</html>
```

7. Explain keyboard and mouse events in JavaScript.

Keyboard and Mouse Events in JavaScript

Keyboard and mouse events in JavaScript are used to respond to user actions performed using keyboard and mouse.

Keyboard Events

Keyboard events occur when user presses or releases keys.

Event	Purpose
onkeydown	Triggered when key is pressed
onkeyup	Triggered when key is released
onkeypress	Triggered when key is pressed

Keyboard Event Example

```
<input type="text" onkeyup="display()">

<script>

  function display() {

    alert("Key Released");

  }

</script>
```

Mouse Events

Mouse events occur when user interacts using mouse.

Event	Purpose
onclick	Triggered when clicked
ondblclick	Triggered on double click
onmouseover	Triggered when mouse moves over element
onmouseout	Triggered when mouse leaves element

Mouse Event Example

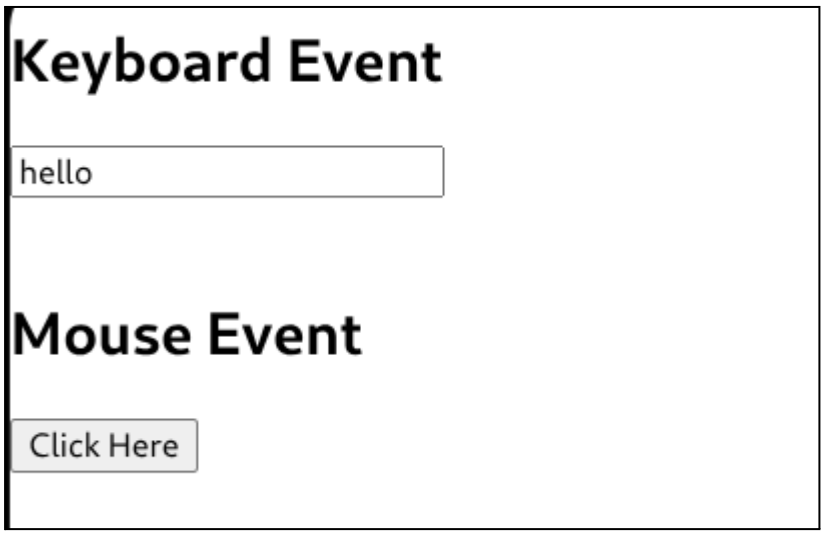
```
<button onclick="show()">
  Click Here
</button>

<script>

  function show() {
```

```
    alert("Button Clicked");
  }
</script>
```

JavaScript Program Using Keyboard and Mouse Events

```
<!DOCTYPE html>
<html>
<head>
  <title>Keyboard and Mouse Events</title>
<script>
  function keyEvent() {
    alert("Key Released");
  }
  function mouseEvent() {
    alert("Mouse Clicked");
  }
</script>
</head>
<body>
  <h2>Keyboard Event</h2>
  <input type="text" onkeyup="keyEvent()">
  <br><br>
  <h2>Mouse Event</h2>
```

```
  <button onclick="mouseEvent()">
    Click Here
  </button>

</body>
</html>
```



8. Define DOM and explain DOM tree structure.

DOM in JavaScript

DOM stands for Document Object Model.

In JavaScript, DOM represents HTML document as a tree structure of objects.

JavaScript uses DOM to:

- Access HTML elements
- Modify webpage content
- Change styles dynamically

Features of DOM

1. Represents webpage as objects
2. Allows dynamic content changes
3. Supports event handling
4. Enables interaction with HTML elements

DOM Tree Structure

DOM organizes HTML elements in hierarchical tree format.

Example HTML:

```
<html>

  <head>
    <title>My Page</title>
  </head>

  <body>

    <h1>Welcome</h1>

    <p>Hello World</p>

  </body>

</html>
```

DOM Tree Representation

```
Document
  |
  html
  |
  |---- head
  |     |
  |     |---- title
  |
  |---- body
        |
        |---- h1
        |
        |---- p
```

Explanation of DOM Nodes

Node	Purpose
Document	Entire webpage
Element Node	HTML tags
Text Node	Content inside tags

Attribute Node	Attributes of elements

DOM Access Example

```
<!DOCTYPE html>
<html>
<head>

  <title>DOM Example</title>

</head>

<body>

  <h1 id="demo">
    Welcome
  </h1>

<script>

  document.getElementById("demo").innerHTML =
  "Hello JavaScript";

</script>

</body>
</html>
```

9. Explain DOM methods such as getElementById and querySelector.

DOM Methods in JavaScript

DOM methods in JavaScript are used to access and manipulate HTML elements.

getElementById()

getElementById() method selects an element using its id.

Syntax:

document.getElementById("idname")

Example:

```
<!DOCTYPE html>
<html>
<head>

  <title>getElementById</title>

</head>

<body>

  <h1 id="demo">
    Welcome
  </h1>

<script>

  document.getElementById("demo").innerHTML =
  "Hello JavaScript";

</script>

</body>
</html>
```

querySelector()

querySelector() method selects the first matching HTML element using CSS selector.

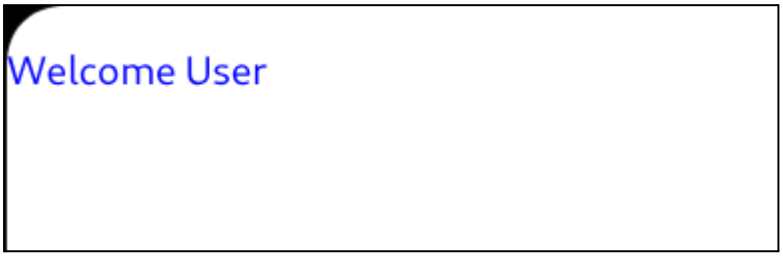
Syntax:

document.querySelector("selector")

Example:

```
<!DOCTYPE html>
```

```
<html>
<head>

  <title>querySelector</title>

</head>

<body>

  <p class="msg">
    Welcome User
  </p>

<script>

  document.querySelector(".msg").style.color =
  "blue";

</script>

</body>
</html>
```



Difference Between getElementById and querySelector

getElementById()	querySelector()
Selects using id only	Selects using CSS selectors
Faster	More flexible
Returns single element	Returns first matching element

10. Explain how JavaScript can change webpage content dynamically.

Dynamic Webpage Content in JavaScript

JavaScript can change webpage content dynamically without reloading the page.

JavaScript uses DOM methods to:

- Change text
- Change images
- Change styles
- Update HTML elements

Methods Used

Method	Purpose
innerHTML	Changes HTML content
document.write()	Displays output
style	Changes CSS styles

Using innerHTML

innerHTML changes the content of HTML element.

Syntax:

```
document.getElementById("id").innerHTML =
"New Content";
```

Example Program

```
<!DOCTYPE html>
<html>
<head>

  <title>Dynamic Content</title>

<script>

  function changeText() {

    document.getElementById("demo").innerHTML =
    "Content Changed Successfully";

  }

</script>
```

```
</head>

<body>

  <h2 id="demo">
    Welcome to JavaScript
  </h2>

  <button onclick="changeText()">
    Change Content
  </button>

</body>
</html>
```

Changing CSS Dynamically

JavaScript can also change styles.

Example:

```
<script>

  document.getElementById("demo").style.color =
  "red";

</script>
```

Changing Image Dynamically

```
<script>

  document.getElementById("img1").src =
  "https://images.unsplash.com/photo-1498050108023-c5249f4df085";

</script>
```

11. Explain dynamic JavaScript. Add a new row to a table dynamically.

Dynamic JavaScript

Dynamic JavaScript refers to changing webpage content dynamically using JavaScript without reloading the webpage.

Dynamic JavaScript is used to:

- Add elements
- Remove elements
- Modify webpage content
- Update tables dynamically

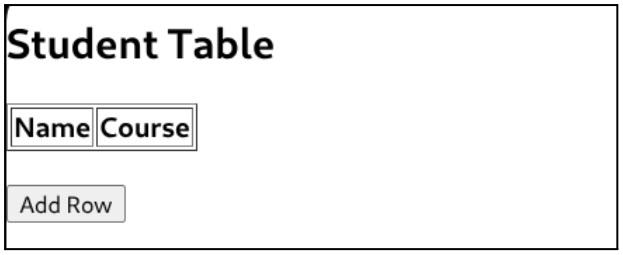
Methods Used

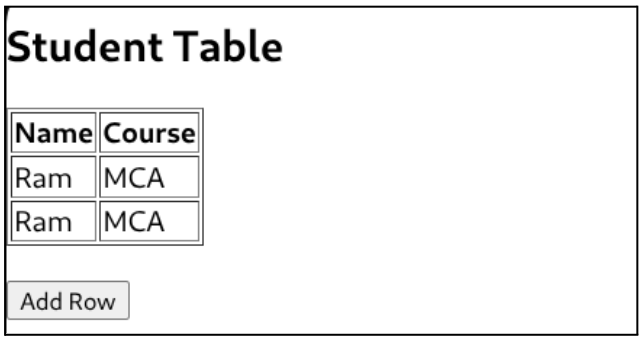
Method	Purpose
createElement()	Creates HTML element
appendChild()	Adds element
insertRow()	Adds new table row
insertCell()	Adds table cell

JavaScript Program to Add New Row Dynamically

```
<!DOCTYPE html>
<html>
<head>

  <title>Dynamic Table Row</title>

<script>

  function addRow() {
```

```
    var table =
    document.getElementById("myTable");

    var row = table.insertRow();

    var cell1 = row.insertCell(0);
    var cell2 = row.insertCell(1);

    cell1.innerHTML = "Ram";
    cell2.innerHTML = "MCA";

  }

</script>

</head>

<body>

  <h2>Student Table</h2>

  <table border="1" id="myTable">

    <tr>

      <th>Name</th>
      <th>Course</th>

    </tr>

  </table>

  <br>

  <button onclick="addRow()">
    Add Row
  </button>

</body>
</html>
```





12. Explain basic browser objects in JavaScript.

Basic Browser Objects in JavaScript

JavaScript provides several browser objects that help interact with the browser window, webpage, user input, and browser information. These objects are part of the Browser Object Model (BOM).

Some commonly used browser objects are:

1. Window Object

The window object is the top-level object in JavaScript. It represents the browser window.

All global variables, functions, and objects automatically become members of the window object.

Common methods:

- alert()
- prompt()
- confirm()
- open()
- close()

Example:

<!DOCTYPE html>
<html>

```
<head>
  <title>Window Object</title>
</head>
<body>

<button onclick="showMessage()">Click</button>

<script>
function showMessage() {
  window.alert("Welcome to JavaScript");
}
</script>

</body>
</html>
```

2. Document Object

The document object represents the HTML document loaded in the browser.

It is used to access and manipulate webpage elements using the DOM.

Common methods:

- getElementById()
- querySelector()
- write()

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Document Object</title>
</head>
<body>

<h2 id="msg">Hello</h2>

<script>
document.getElementById("msg").innerHTML =
  "Welcome to DOM";
</script>
```

```
</body>
</html>
```

JavaScript can dynamically change webpage content using the document object.

3. Navigator Object

The navigator object contains information about the browser.

Properties:

- navigator.appName
- navigator.appVersion
- navigator.platform

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Navigator Object</title>
</head>
<body>

<script>
document.write("Browser Name: " +
  navigator.appName + "<br>");

document.write("Platform: " +
  navigator.platform);
</script>

</body>
</html>
```

4. Location Object

The location object contains information about the current URL.

Common properties:

- location.href
- location.hostname
- location.pathname

Common method:

- location.reload()

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Location Object</title>
</head>
<body>

<button onclick="showURL()">Show URL</button>

<script>
function showURL() {
  alert(location.href);
}
</script>

</body>
</html>
```

5. History Object

The history object stores browser history information.

Methods:

- history.back()
- history.forward()
- history.go()

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>History Object</title>
</head>
<body>

<button onclick="history.back()">
```

```
  Go Back
</button>

</body>
</html>
```

6. Screen Object

The screen object provides information about the user's screen.

Properties:

- screen.width
- screen.height
- screen.colorDepth

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Screen Object</title>
</head>
<body>

<script>
document.write("Screen Width: " +
  screen.width + "<br>");

document.write("Screen Height: " +
  screen.height);
</script>

</body>
</html>
```

13. Explain setTimeout() and setInterval() functions.

setTimeout()

setTimeout() is a JavaScript function used to execute a piece of code or a function once after a specified time delay.

Syntax:

setTimeout(functionName, timeInMilliseconds);

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>setTimeout Example</title>
</head>
<body>

<script>
function message() {
  alert("Hello");
}

setTimeout(message, 3000);
</script>

</body>
</html>
```

In this example, the message is displayed after 3 seconds.

setInterval()

setInterval() is used to repeatedly execute a function after every specified time interval.

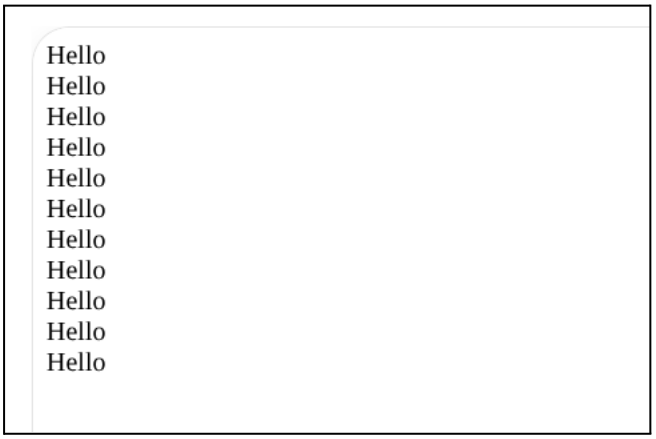
Syntax:

setInterval(functionName, timeInMilliseconds);

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>setInterval Example</title>
</head>
<body>
```

```
<script>
function showTime() {
  document.write("Hello<br>");
}

setInterval(showTime, 1000);
</script>

</body>
</html>
```



In this example, "Hello" is displayed every 1 second.

JavaScript uses setTimeout() and setInterval() for slow movement and repeated execution of elements in dynamic webpages.

14. Explain how JavaScript and DOM can be used to accept marks, calculate total, and display result dynamically.

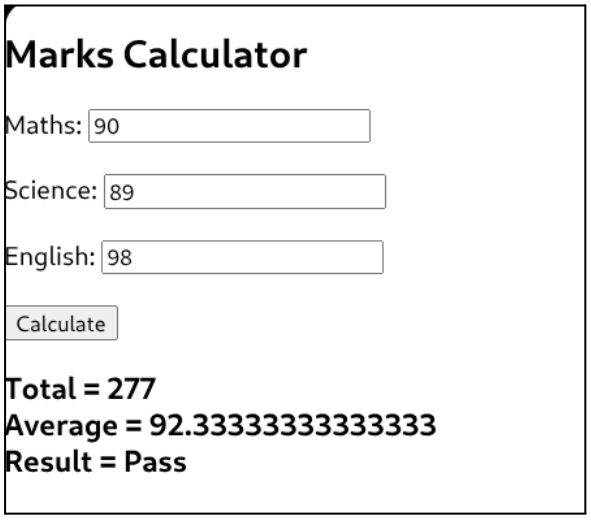
JavaScript and the DOM can be used to dynamically accept marks from the user, calculate the total, and display the result without reloading the webpage.

The DOM allows JavaScript to access HTML elements using methods like getElementById(). The entered marks are read using the value property, calculations are performed, and the result is displayed dynamically using innerHTML.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Result</title>
</head>
```

```
<body>

<h2>Marks Calculator</h2>

Maths:
<input type="text" id="m1"><br><br>

Science:
<input type="text" id="m2"><br><br>

English:
<input type="text" id="m3"><br><br>

<button onclick="calculate()">Calculate</button>

<h3 id="result"></h3>

<script>
function calculate() {

  var m1 = parseInt(document.getElementById("m1").value);
  var m2 = parseInt(document.getElementById("m2").value);
  var m3 = parseInt(document.getElementById("m3").value);

  var total = m1 + m2 + m3;
  var avg = total / 3;

  var res;

  if(avg >= 35) {
    res = "Pass";
  } else {
    res = "Fail";
  }

  document.getElementById("result").innerHTML =
    "Total = " + total +
    "<br>Average = " + avg +
    "<br>Result = " + res;
}
</script>

</body>
</html>
```



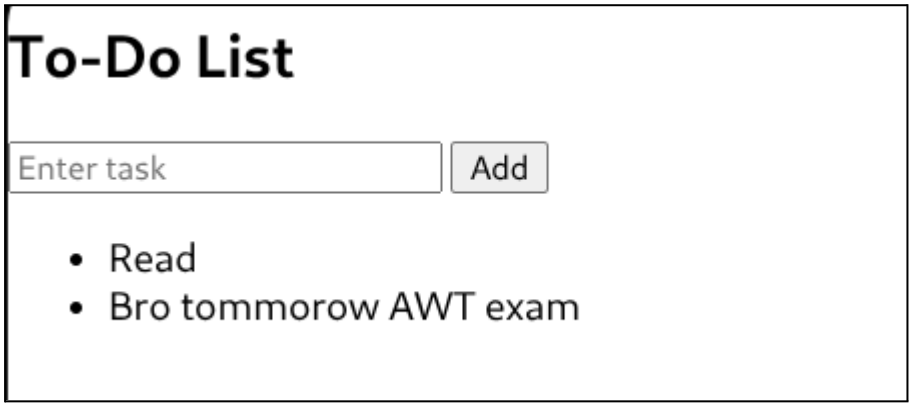
15. Write JavaScript and DOM structure for a to-do list webpage.

A to-do list webpage can be created using HTML, JavaScript, and the DOM. The DOM is used to dynamically add tasks to the webpage.

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>To-Do List</title>
</head>
<body>

<h2>To-Do List</h2>

<input type="text" id="task" placeholder="Enter task">

<button onclick="addTask()">Add</button>

<ul id="list"></ul>

<script>
function addTask() {

  var taskText =
    document.getElementById("task").value;

  var li = document.createElement("li");
```

```
  li.innerHTML = taskText;

  document.getElementById("list").appendChild(li);

  document.getElementById("task").value = "";
}
</script>

</body>
</html>
```



DOM methods used:

- getElementById() → accesses HTML elements
- createElement() → creates new list item
- appendChild() → adds item to webpage dynamically

JavaScript dynamically updates webpage content using the DOM structure.

Module 4 – jQuery & AngularJS

1. Define jQuery and explain its features and advantages.

jQuery

jQuery is a lightweight, fast, and popular JavaScript library designed to simplify JavaScript programming. It follows the principle “write less, do more.” It makes tasks like DOM manipulation, event handling, animations, and AJAX easier.

Features of jQuery

- Lightweight JavaScript library
- Simplifies DOM manipulation
- Supports event handling

- Provides animation and effects
- Simplifies AJAX operations
- Uses CSS selectors for element selection
- Cross-browser compatible
- Supports chaining of methods

Example syntax:

$(selector).action();

Example:

$("p").hide();

This hides all paragraph elements.

Advantages of jQuery

- Reduces amount of JavaScript code
- Easy to learn and use
- Faster web development
- Simplifies complex JavaScript tasks
- Improves webpage interactivity
- Handles browser compatibility issues
- Easy implementation of animations and effects
- Makes AJAX communication simpler

jQuery allows developers to perform common JavaScript tasks using fewer lines of code.

2. Explain different types of jQuery selectors with examples.

jQuery Selectors

jQuery selectors are used to select and manipulate HTML elements. They are based on CSS selectors. All selectors in jQuery start with $().

1. Element Selector

Selects elements based on element name.

Example:

<!DOCTYPE html>

```
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){
  $("button").click(function(){
    $("p").hide();
  });
});
</script>
</head>

<body>

<p>Paragraph 1</p>
<p>Paragraph 2</p>

<button>Hide</button>

</body>
</html>
```

$("p") selects all paragraph elements.

2. ID Selector

Selects an element using its unique id.

Example:

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){
  $("button").click(function(){
    $("#test").hide();
  });
});
</script>
```

```
</head>

<body>

<p id="test">Hello World</p>

<button>Hide</button>

</body>
</html>
```

$("#test") selects the element with id test.

3. Class Selector

Selects elements having a specific class.

Example:

```
<!DOCTYPE html>
<html>
<head>
<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){
  $("button").click(function(){
    $(".demo").hide();
  });
});
</script>
</head>

<body>

<p class="demo">First Paragraph</p>
<p class="demo">Second Paragraph</p>

<button>Hide</button>

</body>
</html>
```

$(".demo") selects all elements with class demo.

3. Explain event handling in jQuery.

Event Handling in jQuery

Event handling in jQuery is used to perform actions when a user interacts with webpage elements such as clicking buttons, moving the mouse, or typing in input fields.

An event represents the precise moment when something happens on a webpage. jQuery provides simple methods to handle these events.

General syntax:

```
$(selector).event(function(){
   // action
});
```

click()

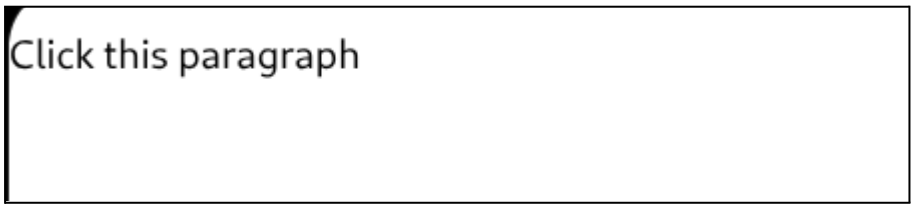
Executed when an element is clicked.

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){

   $("p").click(function(){
      $(this).hide();
   });

});
</script>

</head>
<body>
```

Click this paragraph

</body>
</html>



dblclick()

Executed when an element is double-clicked.

```
$("p").dblclick(function(){
  $(this).hide();
});
```

mouseenter()

Executed when mouse pointer enters an element.

```
$("#p1").mouseenter(function(){
  alert("Mouse entered");
});
```

mouseleave()

Executed when mouse pointer leaves an element.

```
$("#p1").mouseleave(function(){
  alert("Mouse left");
});
```

mousedown()

Executed when mouse button is pressed.

```
$("#p1").mousedown(function(){
  alert("Mouse button pressed");
});
```

mouseup()

Executed when mouse button is released.

```
$("#p1").mouseup(function(){
  alert("Mouse button released");
});
```

hover()

Combination of mouseenter() and mouseleave().

```
$("#p1").hover(
  function(){
    alert("Entered");
  },
  function(){
    alert("Left");
  }
);
```

focus()

Executed when an input field gets focus.

```
$("input").focus(function(){
  $(this).css("background-color", "yellow");
});
```

blur()

Executed when an input field loses focus.

```
$("input").blur(function(){
  $(this).css("background-color", "white");
});
```

on()

Used to attach one or more event handlers.

Example:

```
$("p").on("click", function(){
  $(this).hide();
});
```

jQuery event methods simplify handling user interactions in webpages.

4. Explain jQuery effects such as show(), hide(), toggle(), fadeIn(), fadeOut().

jQuery Effects

jQuery provides effects to show, hide, fade, and animate HTML elements easily.

show()

The show() method displays hidden elements.

Syntax:

```
$(selector).show(speed, callback);
```

Example:

```
$("#btn").click(function(){
  $("#demo").show();
});
```

hide()

The hide() method hides elements.

Syntax:

```
$(selector).hide(speed, callback);
```

Example:

```
$("#btn").click(function(){
  $("#demo").hide();
});
```

toggle()

The toggle() method switches between show and hide.

Example:

```
$("#btn").click(function(){
  $("#demo").toggle();
});
```

If the element is visible, it becomes hidden.
If hidden, it becomes visible.

fadeIn()

The fadeIn() method gradually displays a hidden element.

Syntax:

```
$(selector).fadeIn(speed, callback);
```

Example:

```
$("#btn").click(function(){
  $("#demo").fadeIn();
});
```

fadeOut()

The fadeOut() method gradually hides an element.

Syntax:

```
$(selector).fadeOut(speed, callback);
```

Example:

```
$("#btn").click(function(){
  $("#demo").fadeOut();
});
```

These jQuery effects help create interactive and dynamic webpages.

5. Explain how jQuery changes CSS dynamically.

jQuery can dynamically change CSS properties of HTML elements using the css() method.

Syntax:

$(selector).css(property, value);

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){

  $("button").click(function(){

    $("p").css("color", "red");
    $("p").css("font-size", "25px");

  });

});
</script>

</head>

<body>

<p>This is a paragraph.</p>

<button>Change Style</button>

</body>
</html>
```

Multiple CSS properties can also be changed together.

Example:

```
$("p").css({
  "color": "blue",
  "background-color": "yellow",
  "font-size": "20px"
});
```

jQuery dynamically modifies webpage styles and appearance using DOM manipulation.

6. Explain jQuery DOM methods such as html(), text(), append(), remove().

html()

The html() method is used to get or set HTML content of an element.

Example:

```
$("#demo").html("<b>Hello</b>");
```

This changes the content of the element and applies HTML formatting.

text()

The text() method is used to get or set plain text content.

Example:

```
$("#demo").text("Welcome");
```

This displays only text without HTML formatting.

append()

The append() method adds content at the end of selected elements.

Example:

```
$("p").append(" JavaScript");
```

If the paragraph contains "Learn", the output becomes:

Learn JavaScript

remove()

The remove() method deletes selected HTML elements from the webpage.

Example:

```
$("#demo").remove();
```

This removes the element having id demo.

These methods are used in jQuery for DOM manipulation and dynamic webpage updates.

7. Explain jQuery form validation.

jQuery form validation is used to check whether user input is correct before submitting a form.
It helps prevent empty fields, invalid data, and improves user interaction.

jQuery uses events and selectors to validate form fields dynamically.

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src="https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js"></script>

<script>
$(document).ready(function(){

  $("#myForm").submit(function(){

    var name = $("#name").val();

    if(name == "") {

      alert("Name cannot be empty");
      return false;
```

```
    }

  });

});
</script>

</head>

<body>

<form id="myForm">

Name:
<input type="text" id="name">

<input type="submit" value="Submit">

</form>

</body>
</html>
```

In this example:

- val() gets input field value
- submit() handles form submission event
- If the field is empty, an alert message is displayed
- return false stops form submission

jQuery simplifies form handling and event handling using selectors and methods.

8. Define AngularJS and explain its features.

AngularJS

AngularJS is an open-source JavaScript framework developed by Google for building dynamic single-page web applications. It extends HTML with additional attributes and helps create interactive webpages easily.

AngularJS follows the MVC (Model View Controller) architecture and allows dynamic data binding between HTML and JavaScript.

Features of AngularJS

- Open-source JavaScript framework
- Developed by Google
- Supports single-page applications
- Uses MVC architecture
- Provides two-way data binding
- Supports dependency injection
- Extends HTML with directives
- Simplifies DOM manipulation
- Supports form validation
- Supports reusable components
- Provides routing support
- Easy integration with AJAX and REST services

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>
  <script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
  </script>
</head>

<body>

<div ng-init="name='AngularJS'">

<p>{{ name }}</p>

</div>

</body>
</html>
```

In this example:

- ng-app initializes AngularJS
- ng-init initializes data
- {{ }} displays dynamic data binding

9. Explain AngularJS modules and controllers with syntax.

AngularJS Modules

An AngularJS module is a container used to organize different parts of an AngularJS application such as controllers, directives, and services.

A module is created using the angular.module() function.

Syntax:

```
var app = angular.module("myApp", []);
```

Here:

- "myApp" is module name
- [] represents dependency list

AngularJS Controllers

Controllers are JavaScript functions used to control application data and behavior.

They connect the model and the view.

Syntax:

```
app.controller("myController", function($scope){

  $scope.name = "AngularJS";

});
```

Example:

```
<!DOCTYPE html>
<html ng-app="myApp">

<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body ng-controller="myController">

<h2>{{ name }}</h2>

<script>

var app = angular.module("myApp", []);

app.controller("myController", function($scope){

   $scope.name = "Welcome to AngularJS";

});

</script>

</body>
</html>
```

In this example:

- ng-app defines AngularJS module
- ng-controller defines controller
- $scope transfers data from controller to HTML view
- {{ }} displays data dynamically

10. Explain AngularJS directives such as ng-app, ng-model, ng-repeat.

ng-app

The ng-app directive initializes an AngularJS application and defines the root element of the application.

Example:

<html ng-app="myApp">

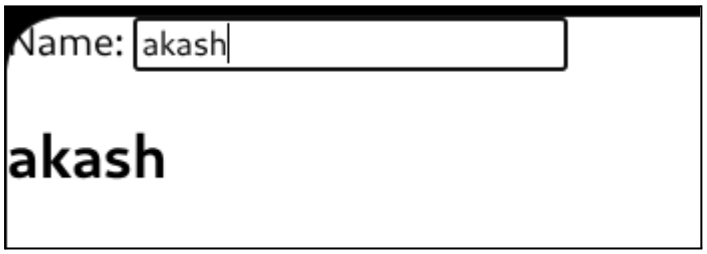
ng-model

The ng-model directive binds HTML form elements with application data.

It provides two-way data binding.

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

Name:
<input type="text" ng-model="name">

<h2>{{ name }}</h2>

</body>
</html>
```



In this example, the entered value is automatically displayed.

ng-repeat

The ng-repeat directive repeats HTML elements for each item in a collection.

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

<ul>
  <li ng-repeat="x in names">
    {{ x }}
  </li>
</ul>

<script>
var app = angular.module("myApp", []);

app.controller("myCtrl", function($scope){

  $scope.names = ["Ram", "Shyam", "Ravi"];

});
</script>

</body>
</html>
```

Here, ng-repeat displays all items in the array repeatedly.

11. Explain two-way data binding in AngularJS.

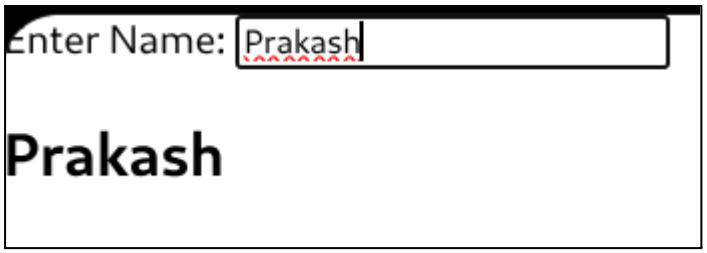
Two-way data binding in AngularJS automatically synchronizes data between the model and the view.

When data in the model changes, the view updates automatically.
Similarly, when the user changes data in the view, the model also updates automatically.

AngularJS mainly uses the ng-model directive for two-way data binding.

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

Enter Name:
<input type="text" ng-model="name">

<h2>{{ name }}</h2>

</body>
</html>
```



In this example:

- ng-model="name" binds input field with variable name
- {{ name }} displays the value dynamically
- Any change in the textbox immediately updates the displayed output

Thus, the model and view remain synchronized automatically.

12. Explain AngularJS form handling and validation.

AngularJS form handling is used to manage user input and process forms dynamically.
AngularJS provides built-in validation features using directives.

Common validation directives:

- required
- ng-model
- ng-minlength
- ng-maxlength
- ng-pattern

AngularJS automatically tracks form state such as:

- Valid
- Invalid
- Touched
- Untouched

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

<form name="myForm">

Name:
<input type="text"
    name="uname"
    ng-model="uname"
    required>
```

```
<span ng-show="myForm.uname.$error.required">
  Name is required
</span>

<br><br>

Email:
<input type="email"
    name="email"
    ng-model="email"
    required>

<span ng-show="myForm.email.$error.email">
  Invalid email
</span>

</form>

</body>
</html>
```

In this example:

- ng-model binds form data
- required checks empty fields
- type="email" validates email format
- ng-show displays validation messages dynamically

AngularJS performs validation automatically without writing large amounts of JavaScript code.

13. Explain AngularJS filters with examples.

AngularJS filters are used to format and transform data before displaying it in the view.

Filters are applied using the pipe symbol (|).

Syntax:

{{ expression | filterName }}

Common AngularJS filters:

- uppercase
- lowercase
- currency
- date
- filter
- orderBy

uppercase Filter

Converts text to uppercase.

Example:

{{ "angularjs" | uppercase }}

Output:

ANGULARJS

lowercase Filter

Converts text to lowercase.

Example:

{{ "HELLO" | lowercase }}

Output:

hello

currency Filter

Formats a number as currency.

Example:

{{ 5000 | currency }}

Output:

$5,000.00

date Filter

Formats date values.

Example:

```
<p>{{ today | date:'dd/MM/yyyy' }}</p>
```

orderBy Filter

Sorts an array.

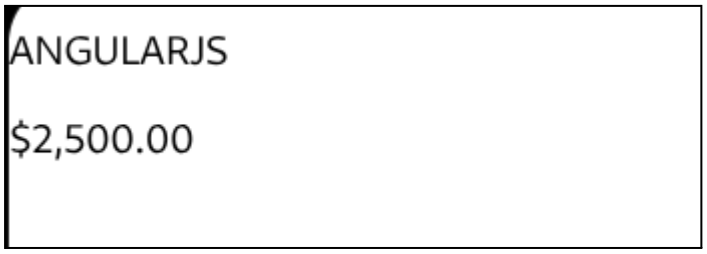
Example:

```
<li ng-repeat="x in names | orderBy">
  {{ x }}
</li>
```

Example Program:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

<p>{{ "angularjs" | uppercase }}</p>

<p>{{ 2500 | currency }}</p>

</body>
```

</html>



14. Explain how jQuery can be used for notification panel, highlighting menu, and validation.

jQuery can be used to create interactive webpages by dynamically handling notifications, menu highlighting, and form validation using selectors, events, and CSS manipulation.

Notification Panel

A notification panel can be shown or hidden using jQuery effects like show(), hide(), or toggle().

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">
</script>

<script>
$(document).ready(function(){

  $("#btn").click(function(){
    $("#panel").toggle();
  });

});
</script>

</head>

<body>
```

```
<button id="btn">Notifications</button>

<div id="panel" style="display:none;">
  New Message Received
</div>

</body>
</html>
```

Highlighting Menu

jQuery changes CSS dynamically to highlight menu items when the mouse moves over them.

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">
</script>

<script>
$(document).ready(function(){

  $("li").hover(function(){

    $(this).css("background-color", "yellow");

  },

  function(){

    $(this).css("background-color", "white");

  });

});
</script>
```

```
</head>

<body>

<ul>
  <li>Home</li>
  <li>About</li>
  <li>Contact</li>
</ul>

</body>
</html>
```

Form Validation

jQuery validates user input before form submission.

Example:

```
<!DOCTYPE html>
<html>
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/jquery/3.7.1/jquery.min.js">
</script>

<script>
$(document).ready(function(){

  $("#myForm").submit(function(){

    var name = $("#name").val();

    if(name == "") {

      alert("Name cannot be empty");
      return false;

    }

  });
```

```
});
</script>

</head>

<body>

<form id="myForm">

Name:
<input type="text" id="name">

<input type="submit" value="Submit">

</form>

</body>
</html>
```

15. Explain how AngularJS can display product details, update quantity, and calculate total.

AngularJS can dynamically display product details, update quantity using two-way data binding, and automatically calculate the total amount.

The ng-model directive binds input values, and expressions inside {{ }} update automatically whenever data changes.

Example:

```
<!DOCTYPE html>
<html ng-app="">
<head>

<script src=
"https://ajax.googleapis.com/ajax/libs/angularjs/1.8.2/angular.min.js">
</script>

</head>

<body>

<h2>Product Details</h2>
```

```
Product Name:
<input type="text" ng-model="pname">
<br><br>

Price:
<input type="number" ng-model="price">
<br><br>

Quantity:
<input type="number" ng-model="qty">
<br><br>

<h3>Product: {{ pname }}</h3>

<h3>Total Amount: {{ price * qty }}</h3>

</body>
</html>
```

In this example:

- ng-model binds product data
- Quantity updates dynamically
- Total amount is calculated automatically using AngularJS expressions
- Any change in price or quantity immediately updates the total due to two-way data binding

Module 5 – AJAX, JSON, Web 2.0

1. Define AJAX and explain its features and advantages.

AJAX

AJAX stands for Asynchronous JavaScript and XML.
It is a web development technique used to create fast and interactive web applications by exchanging small amounts of data with the server without reloading the entire webpage.

AJAX uses technologies such as:

- HTML/XHTML
- CSS
- JavaScript
- DOM
- XMLHttpRequest

Features of AJAX

- Asynchronous communication
- Partial page updates
- Faster response time
- Uses JavaScript and DOM
- Reduces full page reloads
- Supports data exchange using XML, JSON, HTML, or text
- Improves interactivity of webpages
- Uses XMLHttpRequest object for server communication

Advantages of AJAX

- Improves user experience
- Faster webpage interaction
- Reduces server load
- Reduces bandwidth usage
- Updates only required parts of webpage
- Makes web applications more responsive
- Provides desktop-like behavior in browser applications
- Minimizes waiting time for users

AJAX enables dynamic communication between client and server without refreshing the entire webpage.

2. Explain the working steps of AJAX.

Working Steps of AJAX

1. The user performs an action on the webpage such as clicking a button, typing text, or submitting a form.
2. JavaScript captures the event and sends the request to the AJAX engine instead of directly reloading the webpage.
3. The AJAX engine creates an XMLHttpRequest object.
4. The request is sent asynchronously to the web server.
5. The server processes the request and prepares the response data.
6. The server sends the response back to the AJAX engine in formats such as XML, JSON, HTML, or text.
7. JavaScript receives the response and processes the data.
8. Using the DOM, only the required part of the webpage is updated dynamically without refreshing the entire page.

3. Explain XMLHttpRequest object and its methods. Write basic AJAX syntax.

XMLHttpRequest Object

The XMLHttpRequest object is used in AJAX to send and receive data from a web server asynchronously without reloading the webpage.

It allows JavaScript to:

- Send HTTP requests to server
- Receive server responses
- Update webpage dynamically using DOM

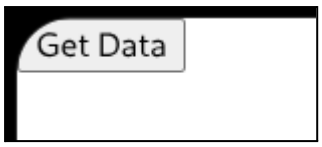
Methods of XMLHttpRequest

Method	Purpose
open()	Initializes request
send()	Sends request to server
setRequestHeader()	Sets HTTP header values
abort()	Cancels request
getResponseHeader()	Gets specific response header

getAllResponseHeaders()	Gets all response headers

Basic AJAX Syntax

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Example</title>
</head>
<body>

<button onclick="loadData()">Get Data</button>

<div id="demo"></div>

<script>

function loadData() {

  var xhttp = new XMLHttpRequest();

  xhttp.onreadystatechange = function() {

    if(this.readyState == 4 && this.status == 200) {

      document.getElementById("demo").innerHTML =
        this.responseText;

    }

  };

  xhttp.open("GET", "data.txt", true);

  xhttp.send();
}

</script>

</body>
</html>
```



In this example:

- open() prepares request
- send() sends request
- responseText gets server response
- DOM updates webpage dynamically without refresh

4. Explain the use of fetch() in AJAX.

The fetch() function is a modern JavaScript method used in AJAX to send requests to a server and receive responses asynchronously.

It is used as an alternative to the XMLHttpRequest object and provides a simpler syntax.

Features:

- Sends HTTP requests
- Receives server responses asynchronously
- Supports GET and POST methods
- Returns a Promise
- Used for dynamic webpage updates without reloading

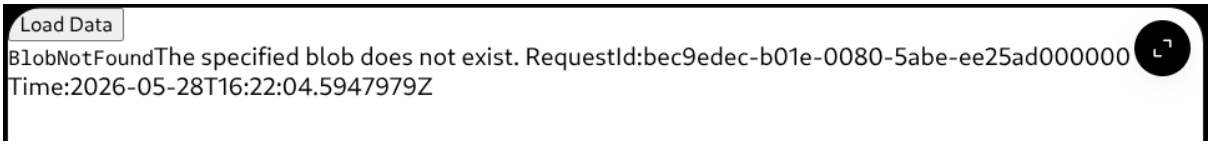
Syntax:

```
fetch(url)
.then(response => response.text())
.then(data => {
   // process data
});
```

Example:

```
<!DOCTYPE html>
<html>
<head>
   <title>Fetch Example</title>
</head>
<body>
```

```
<button onclick="loadData()">Load Data</button>

<div id="demo"></div>

<script>

function loadData() {

  fetch("data.txt")

  .then(response => response.text())

  .then(data => {

    document.getElementById("demo").innerHTML = data;

  });

}

</script>

</body>
</html>
```



In this example:

- fetch() sends request to server
- response.text() reads response data
- DOM updates webpage dynamically without refreshing the page

5. Explain how AJAX is used in form submission.

AJAX is used in form submission to send form data to the server asynchronously without reloading the entire webpage.

JavaScript collects form data and sends it using XMLHttpRequest or fetch(). The server processes the request and returns a response, which is displayed dynamically using the DOM.

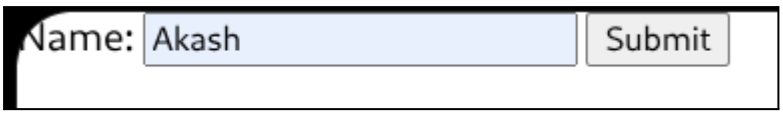
Steps:

1. User fills the form
2. JavaScript captures submit event
3. AJAX sends data to server
4. Server processes request
5. Response is returned
6. Webpage updates dynamically without refresh

Example using XMLHttpRequest:

```
<!DOCTYPE html>
<html>
<head>
  <title>AJAX Form</title>
</head>
<body>

<form id="myForm">

Name:
<input type="text" id="name">

<input type="button"
    value="Submit"
    onclick="sendData()">

</form>

<div id="result"></div>

<script>

function sendData() {

  var name =
    document.getElementById("name").value;

  var xhttp = new XMLHttpRequest();

  xhttp.onreadystatechange = function() {

    if(this.readyState == 4 &&
      this.status == 200) {

       document.getElementById("result")
```

```
        .innerHTML = this.responseText;

    }

  };

  xhttp.open("GET",
    "server.php?name=" + name,
    true);

  xhttp.send();
}

</script>

</body>
</html>
```



AJAX form submission improves speed and user experience by avoiding full page reloads.

6. What is AJAX live search? Explain implementation steps.

AJAX live search is a technique where search results are displayed dynamically while the user types, without reloading the webpage.

AJAX sends requests to the server asynchronously and updates matching results instantly. This improves speed and user experience.

Implementation Steps

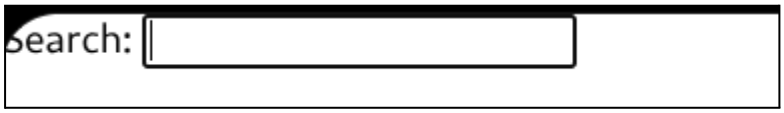
1. Create a search textbox in HTML.
2. Capture user input using JavaScript events such as keyup.
3. Create an AJAX request using XMLHttpRequest or fetch().
4. Send typed text to the server asynchronously.
5. Server processes the request and searches matching data.
6. Server returns search results.
7. JavaScript receives response and updates webpage dynamically using DOM.

Example:

```
<!DOCTYPE html>
```

```
<html>
<head>
  <title>AJAX Live Search</title>
</head>
<body>

Search:
<input type="text" onkeyup="showResult(this.value)">

<div id="result"></div>

<script>

function showResult(str) {

  if(str.length == 0) {

    document.getElementById("result").innerHTML = "";
    return;

  }

  var xhttp = new XMLHttpRequest();

  xhttp.onreadystatechange = function() {

    if(this.readyState == 4 &&
      this.status == 200) {

       document.getElementById("result")
         .innerHTML = this.responseText;

    }

  };

  xhttp.open("GET",
    "search.php?q=" + str,
    true);

  xhttp.send();
}

</script>
```

```
</body>
</html>
```



7. Define JSON and explain its key–value pair structure and rules.

JSON

JSON stands for JavaScript Object Notation.
It is a lightweight data interchange format used to store and exchange data between client and server.

JSON is easy for humans to read and write and easy for machines to parse and generate.

Key–Value Pair Structure

JSON data is represented as key–value pairs.

Syntax:

```
{
  "name": "Ram",
  "age": 21,
  "city": "Mangalore"
}
```

Here:

- "name", "age", "city" are keys
- "Ram", 21, "Mangalore" are values

Rules of JSON

- Data is written in key–value pairs
- Keys must be enclosed in double quotes
- Values can be:
    - String
    - Number
    - Boolean

    - Array
    - Object
    - null
- Key and value are separated by colon (:)
- Multiple pairs are separated by commas
- JSON objects are enclosed within { }
- Arrays are enclosed within [ ]

Example with array:

```
{
  "students": ["Ram", "Shyam", "Ravi"]
}
```

8. Explain JSON objects and arrays with examples.

JSON Objects

A JSON object is a collection of key–value pairs enclosed within curly braces { }.

Example:

```
{
  "name": "Ravi",
  "age": 22,
  "city": "Mysore"
}
```

Here:

- name, age, and city are keys
- "Ravi", 22, and "Mysore" are values

Values can be strings, numbers, booleans, arrays, objects, or null.

JSON Arrays

A JSON array is an ordered collection of values enclosed within square brackets [ ].

Example:

```
{
  "colors": ["Red", "Green", "Blue"]
}
```

Here:

- colors is the key
- Array contains multiple values

JSON Object with Array

Example:

```
{
  "student": {
    "name": "Ram",
    "marks": [85, 90, 88]
  }
}
```

In this example:

- student is a JSON object
- marks is a JSON array inside the object

9. Explain how JSON data is accessed in JavaScript.

JSON data in JavaScript is accessed using dot notation or bracket notation after converting the JSON string into a JavaScript object.

JSON is commonly converted using JSON.parse().

Example:

```
<!DOCTYPE html>
<html>
<head>
  <title>JSON Access</title>
</head>
<body>

<script>

var data = `{
  "name": "Ram",
  "age": 21,
  "city": "Mangalore"
```

```
}`;

var obj = JSON.parse(data);

document.write(obj.name + "<br>");
document.write(obj.age + "<br>");
document.write(obj["city"]);

</script>

</body>
</html>
```

In this example:

- JSON.parse() converts JSON string into JavaScript object
- obj.name uses dot notation
- obj["city"] uses bracket notation

Accessing JSON array values:

```
var student = {
  "marks": [85, 90, 95]
};

document.write(student.marks[0]);
```

Here, array elements are accessed using index values.

10. Define Web 2.0 and explain its main features.

Web 2.0

Web 2.0 refers to the second generation of web development that focuses on user interaction, dynamic content, collaboration, and rich internet applications.

It allows users not only to view information but also to create, share, and interact with content online. AJAX played an important role in the growth of Web 2.0 applications.

Main Features of Web 2.0

- Dynamic and interactive webpages
- User-generated content
- Rich user interface
- Faster and responsive applications
- Asynchronous communication using AJAX
- Social networking support
- Collaboration and sharing
- Real-time updates without page reload
- Use of APIs and web services
- Improved user experience

Examples:

- Google Maps
- Gmail
- Facebook
- YouTube

These applications use dynamic updates and asynchronous communication for better interactivity.

11. Explain the role of Web 2.0 in modern web applications.

Web 2.0 plays an important role in modern web applications by making websites interactive, dynamic, and user-centered.

Earlier web applications were mostly static, but Web 2.0 introduced technologies that allow webpages to update dynamically without reloading completely.

Roles of Web 2.0 in modern web applications:

- Enables dynamic content updates
- Supports real-time interaction
- Improves user experience
- Allows asynchronous communication using AJAX
- Supports user-generated content
- Enables collaboration and social networking
- Provides rich internet applications
- Reduces full page reloads
- Makes applications faster and more responsive
- Supports integration of APIs and web services

Applications like Gmail, Google Maps, and Google Suggest use Web 2.0 technologies to provide interactive and responsive interfaces.

12. Explain asynchronous communication in web applications.

Asynchronous communication in web applications is a technique where data is exchanged between client and server without stopping or reloading the entire webpage.

In traditional web applications, the browser waits for the server response and reloads the whole page. In asynchronous communication, requests are sent in the background while the user continues interacting with the application.

AJAX mainly uses asynchronous communication through the XMLHttpRequest object or fetch().

Working:

1. User performs an action
2. JavaScript sends request asynchronously
3. Server processes request
4. Response is returned
5. Only required part of webpage is updated dynamically

Advantages:

- Faster response
- Better user experience
- Reduced page reloads
- Reduced bandwidth usage
- More interactive applications

Applications like Gmail and Google Maps use asynchronous communication for dynamic updates without refreshing the webpage.

13. Explain how AJAX supports Web 2.0 features.

AJAX supports Web 2.0 features by enabling dynamic and interactive web applications through asynchronous communication between client and server.

AJAX allows webpages to exchange small amounts of data without reloading the entire page, which improves responsiveness and user experience.

Ways AJAX supports Web 2.0:

- Enables partial page updates

- Provides real-time interaction
- Reduces full page reloads
- Improves speed and responsiveness
- Supports rich internet applications
- Allows asynchronous communication
- Enhances user experience
- Supports dynamic content loading

Applications such as Gmail, Google Maps, and Google Suggest use AJAX to dynamically update webpage content while users continue interacting with the application.

AJAX combines technologies such as:

- HTML/XHTML
- CSS
- JavaScript
- DOM
- XMLHttpRequest

These technologies help Web 2.0 applications behave more like desktop applications inside the browser.

14. Explain how AJAX and JSON can be used to fetch and display product details dynamically.

AJAX and JSON can be used together to fetch product details from the server and display them dynamically without reloading the webpage.

AJAX sends requests asynchronously, and the server returns product data in JSON format. JavaScript then parses the JSON data and updates the webpage using the DOM.

Steps:

1. User performs an action
2. AJAX sends request to server
3. Server returns product details in JSON format
4. JavaScript reads JSON data
5. DOM updates webpage dynamically

Example:

<!DOCTYPE html>
<html>

```
<head>
  <title>Product Details</title>
</head>
<body>

<button onclick="loadProduct()">
  Load Product
</button>

<div id="demo"></div>

<script>

function loadProduct() {

  var xhttp = new XMLHttpRequest();

  xhttp.onreadystatechange = function() {

    if(this.readyState == 4 &&
      this.status == 200) {

      var product =
        JSON.parse(this.responseText);

      document.getElementById("demo")
        .innerHTML =
        "Product: " + product.name +
        "<br>Price: " + product.price +
        "<br>Brand: " + product.brand;

    }

  };

  xhttp.open("GET", "product.json", true);

  xhttp.send();
}

</script>

</body>
</html>
```

Example product.json file:

```
{
  "name": "Laptop",
  "price": 50000,
  "brand": "HP"
}
```

In this example:

- AJAX fetches product data
- JSON.parse() converts JSON text into JavaScript object
- DOM dynamically displays product details without refreshing the webpage

15. Explain how AJAX, JSON, and Web 2.0 help in live order status updates and notifications.

AJAX, JSON, and Web 2.0 together help web applications provide live order status updates and notifications without reloading the webpage.

AJAX sends asynchronous requests to the server at regular intervals or whenever needed. The server returns updated order information in JSON format. JavaScript processes the JSON data and dynamically updates the webpage using the DOM.

Working:

1. User places an order
2. AJAX sends request to server
3. Server checks latest order status
4. Updated data is returned in JSON format
5. JavaScript reads JSON data
6. DOM updates order status and notifications dynamically

Example JSON response:

```
{
  "orderId": 101,
  "status": "Shipped",
  "message": "Your order has been shipped"
```

```
}
```

Example AJAX code:

```
function checkStatus() {

  fetch("order.json")

  .then(response => response.json())

  .then(data => {

    document.getElementById("status")
      .innerHTML = data.status;

    document.getElementById("msg")
      .innerHTML = data.message;

  });

}
```

Benefits:

- Real-time updates
- Faster response
- Better user experience
- No full page reload
- Interactive Web 2.0 application

Applications like online shopping and food delivery systems use these technologies for live tracking and instant notifications.